SOFTWARE DESIGN PATTERNS
FINAL PROJECT REPORT

Restaurant Ordering System

A Java console application integrating seven design patterns:
Factory Method, Decorator, Observer, Strategy, Builder, Bridge and Facade

Team member	Role	Patterns implemented
Yeskendir Abraimov	Student 1 - menu & meals	Factory Method, Decorator
Dias Zhumakhan	Student 2 - order processing	Observer, Strategy
Kuandyk Koshanov	Student 3 - architecture & output	Builder, Bridge, Facade

2026

Contents

1. Introduction

1.1 Problem statement · 1.2 Goals and objectives · 1.3 Technologies · 1.4 Distribution of work

2. Body: design and implementation

2.0 Architecture overview · 2.1 Factory Method · 2.2 Decorator · 2.3 Observer · 2.4 Strategy · 2.5 Builder · 2.6 Bridge · 2.7 Facade

2.8 How the patterns work together · 2.9 Application screenshots · 2.10 Building and running

3. Conclusion

4. Further work

References

Appendix A. Full source code

1. Introduction

1.1 Problem statement

A restaurant has to handle a constant stream of orders that differ in many ways: a guest can eat at a table or order delivery, every meal can be customised with extra toppings, several people (the kitchen, the customer, the cashier) must learn about each change of the order, and the final price depends on promotions and on the payment method. If all of this is written as one large class with many `if/else` branches, the code becomes hard to read, hard to test and, most importantly, hard to extend - adding a new topping, a new discount or a new receipt format would mean editing code that already works.

The goal of this project is to show how classic object-oriented design patterns solve exactly these problems. We built a **Restaurant Ordering System** ("Design Patterns Bistro") - a Java console application in which a user can browse the menu, create dine-in and delivery orders with customised meals, follow the order through the kitchen, pay for it with different discounts and payment methods, and print receipts in different formats.

1.2 Goals and objectives

- Apply at least six distinct design patterns from the course inside **one cohesive project** (we use seven).
- Each team member individually implements at least two patterns.
- Make the patterns cooperate: objects produced by one pattern are consumed by another (e.g. meals created by the Factory Method are wrapped by Decorators, collected by the Builder, priced by a Strategy and printed through the Bridge).
- Provide a working application with an interactive menu and a scripted demo that proves every pattern is operational.
- Document the design with UML class and sequence diagrams.

1.3 Technologies

Java (source level 11, tested with OpenJDK 21), no external libraries; console (CLI) application with an interactive menu and a `--demo` mode; UML diagrams generated with Graphviz.

1.4 Distribution of work

Student	Patterns	Package(s)	Main classes
Yeskendir Abraimov	Factory Method Decorator	`restaurant.factory` `restaurant.decorator`	Meal, BaseMeal, MealFactory, Burger/Pizza/Salad/DrinkFactory, ToppingDecorator, ExtraCheese, Bacon, SpicySauce, ExtraIce, Topping
Dias Zhumakhan	Observer Strategy	`restaurant.observer` `restaurant.strategy`	OrderSubject, OrderObserver, KitchenDisplay, CustomerNotifier, CashierDashboard, DiscountStrategy (+3), PaymentStrategy (+3)
Kuandyk Koshanov	Builder Bridge Facade	`restaurant.builder` `restaurant.bridge` `restaurant.facade`	Order, Order.Builder, Receipt, ShortReceipt, DetailedReceipt, ReceiptRenderer, ConsoleRenderer, HtmlRenderer, RestaurantFacade

In total the project contains **7 distinct design patterns** (the requirement is at least 6): two patterns by Yeskendir Abraimov, two by Dias Zhumakhan and three by Kuandyk Koshanov. The console client `Main` was written together.

2. Body: design and implementation

2.0 Architecture overview

The project is divided into one package per pattern, which makes the authorship and responsibility of every student visible in the code structure:

```
RestaurantOrderingSystem/
+-- src/restaurant/
|   +-- Main.java                 console client (menu + demo)
|   +-- factory/    (Yeskendir Abraimov)    Meal, BaseMeal, MealCategory, MealFactory + 4 concrete factories, 4 meals
|   +-- decorator/  (Yeskendir Abraimov)    ToppingDecorator, ExtraCheese, Bacon, SpicySauce, ExtraIce, Topping
|   +-- observer/   (Dias Zhumakhan)         OrderSubject, OrderObserver, OrderStatus, KitchenDisplay,
|   |                             CustomerNotifier, CashierDashboard
|   +-- strategy/   (Dias Zhumakhan)         DiscountStrategy + 3 discounts, PaymentStrategy + 3 payment methods
|   +-- builder/    (Kuandyk Koshanov)      Order with nested Order.Builder
|   +-- bridge/     (Kuandyk Koshanov)      Receipt, ShortReceipt, DetailedReceipt, ReceiptRenderer,
|   |                             ConsoleRenderer, HtmlRenderer
|   +-- facade/     (Kuandyk Koshanov)      RestaurantFacade
+-- uml/                          Graphviz sources of the UML diagrams
+-- build.sh / build.bat, run.sh / run.bat, README.md
```

The console client talks **only** to `RestaurantFacade`. The facade coordinates the subsystem: it asks a factory for a meal, wraps it with decorators, builds and registers an order, subscribes the observers, applies a discount and a payment strategy and finally prints a receipt through the bridge. Figure 1 shows the whole system; colours mark the author of each class (blue - Yeskendir Abraimov, green - Dias Zhumakhan, orange - Kuandyk Koshanov).

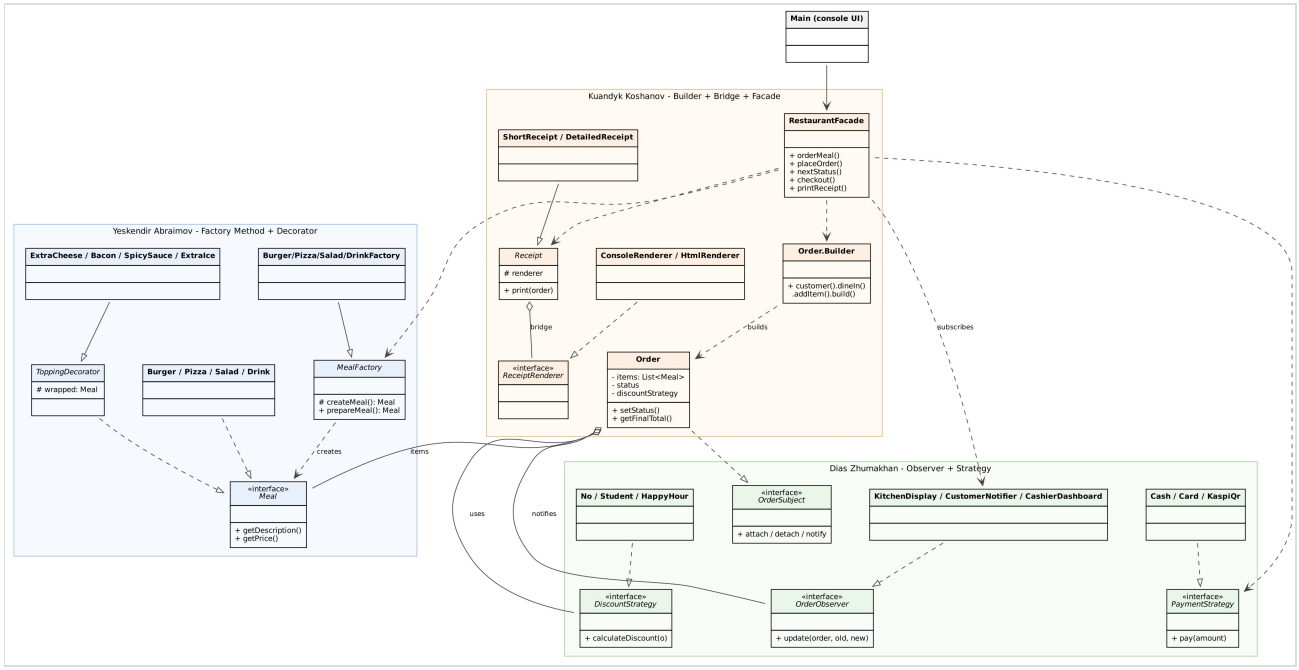


Figure 1. Overall UML class diagram of the Restaurant Ordering System

2.1 Factory Method implemented by Yeskendir Abraimov

Intent. Define an interface for creating an object, but let subclasses decide which class to instantiate.

Why it is used here. The restaurant has several kitchen stations, each producing its own kind of meal. The client never calls `new Burger()` - constructors of the concrete meals are package-private - so the creation logic is in one place and adding a new meal (e.g. Sushi) means adding one product and one factory class, without changing existing code (Open/Closed principle). The common workflow `prepareMeal()` is written once in the abstract creator and calls the factory method `createMeal()` implemented by subclasses.

Pattern role	Class in the project
Product	`Meal` (interface), `BaseMeal`
Concrete products	`Burger`, `Pizza`, `Salad`, `Drink`
Creator	`MealFactory` - abstract `createMeal()`, template `prepareMeal()`
Concrete creators	`BurgerFactory`, `PizzaFactory`, `SaladFactory`, `DrinkFactory`

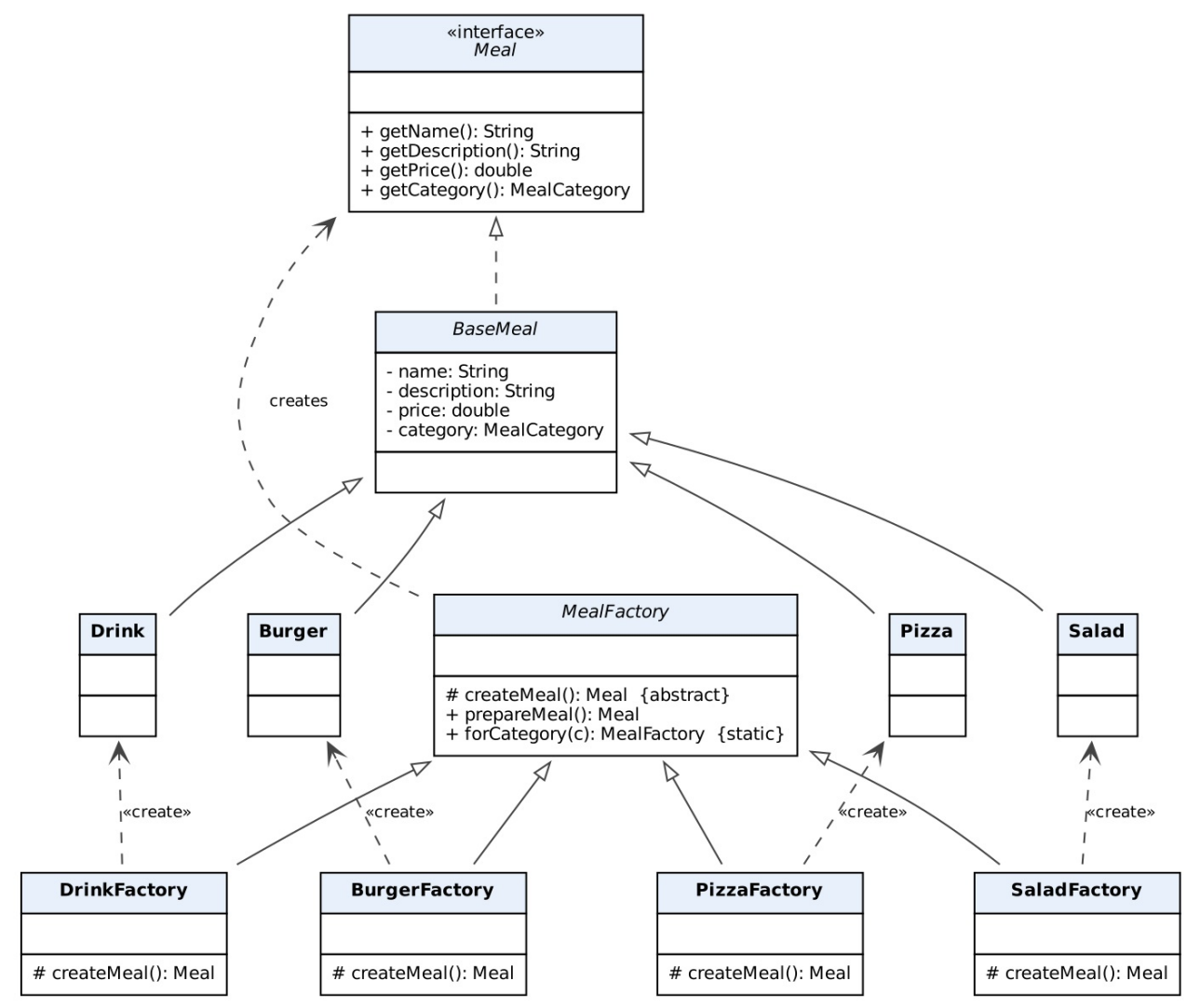


Figure 2. UML class diagram - Factory Method

Key implementation.

factory/MealFactory.java

```java
package restaurant.factory;

/**
 * FACTORY METHOD pattern - Creator.
 *
 * createMeal() is the factory method: each subclass decides which concrete
 * Meal to instantiate. prepareMeal() is the common workflow that every
 * kitchen station follows and that relies on the factory method.
 *
 * Author: Yeskendir Abraimov (Student 1)
 */
public abstract class MealFactory {

    /** The factory method - implemented by concrete creators. */
    protected abstract Meal createMeal();

    /** Common business logic that uses the product created by the subclass. */
    public Meal prepareMeal() {
        Meal meal = createMeal();
        System.out.println("  [Factory] " + getClass().getSimpleName()
                + " prepared: " + meal.getDescription());
        return meal;
    }

    /** Returns the right concrete creator for the requested category. */
    public static MealFactory forCategory(MealCategory category) {
        switch (category) {
            case BURGER: return new BurgerFactory();
            case PIZZA:  return new PizzaFactory();
            case SALAD:  return new SaladFactory();
            case DRINK:  return new DrinkFactory();
            default: throw new IllegalArgumentException("Unknown meal: " + category);
        }
    }
}
```

factory/BurgerFactory.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete creator. Author: Yeskendir Abraimov (Student 1) */
public class BurgerFactory extends MealFactory {
    @Override
    protected Meal createMeal() {
        return new Burger();
    }
}
```

factory/Burger.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete product. Author: Yeskendir Abraimov (Student 1) */
class Burger extends BaseMeal {
    Burger() {
        super("Burger", "Beef Burger", 2500, MealCategory.BURGER);
    }
}
```

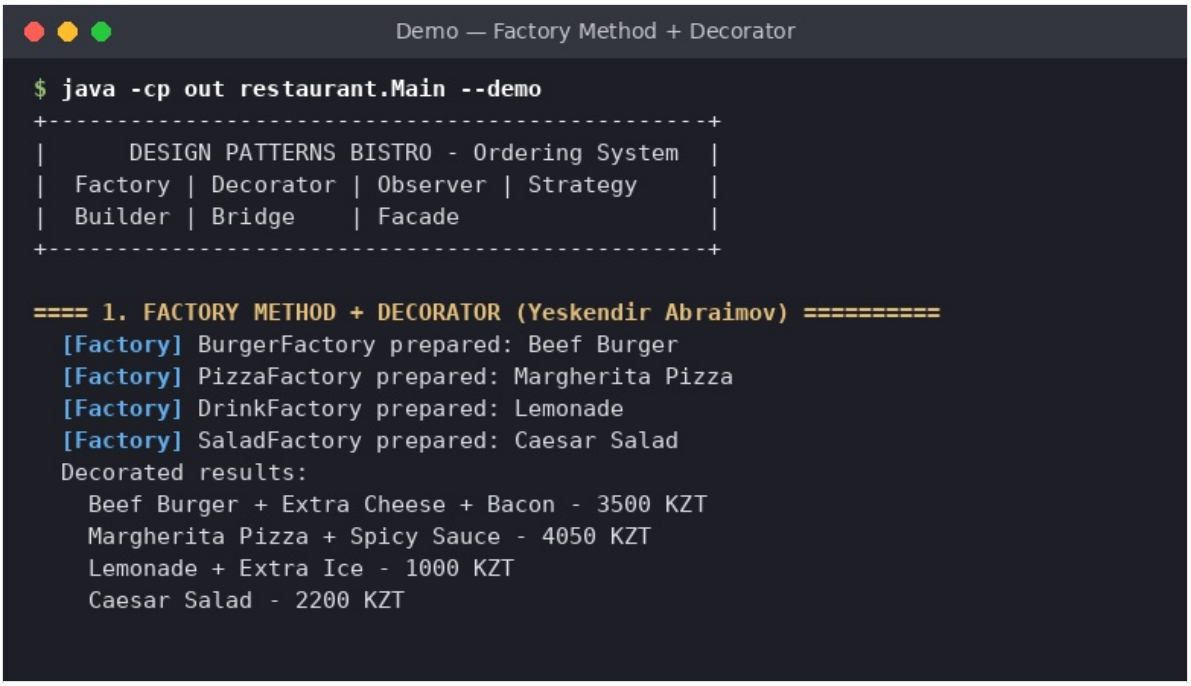


Figure 3. Factory Method in action - each concrete factory prepares its meal; Decorator adds toppings

2.2 Decorator implemented by Yeskendir Abraimov

Intent. Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

Why it is used here. Any meal can get any combination of toppings. Creating subclasses such as `BurgerWithCheeseAndBacon` would lead to a class explosion. Instead each topping is a decorator that implements the same `Meal` interface, holds a reference to the wrapped meal and adds its own price and description. Decorators can be stacked in any order at run time: `new Bacon(new ExtraCheese(burger))` costs 2500 + 400 + 600 = 3500 KZT.

Pattern role	Class in the project
Component	`Meal`
Concrete component	meals created by the Factory Method (`Burger`, `Pizza`, ...)
Decorator	`ToppingDecorator` (abstract, field `wrapped`)
Concrete decorators	`ExtraCheese`, `Bacon`, `SpicySauce`, `ExtraIce`
Helper	`Topping` enum - applies a decorator by menu choice

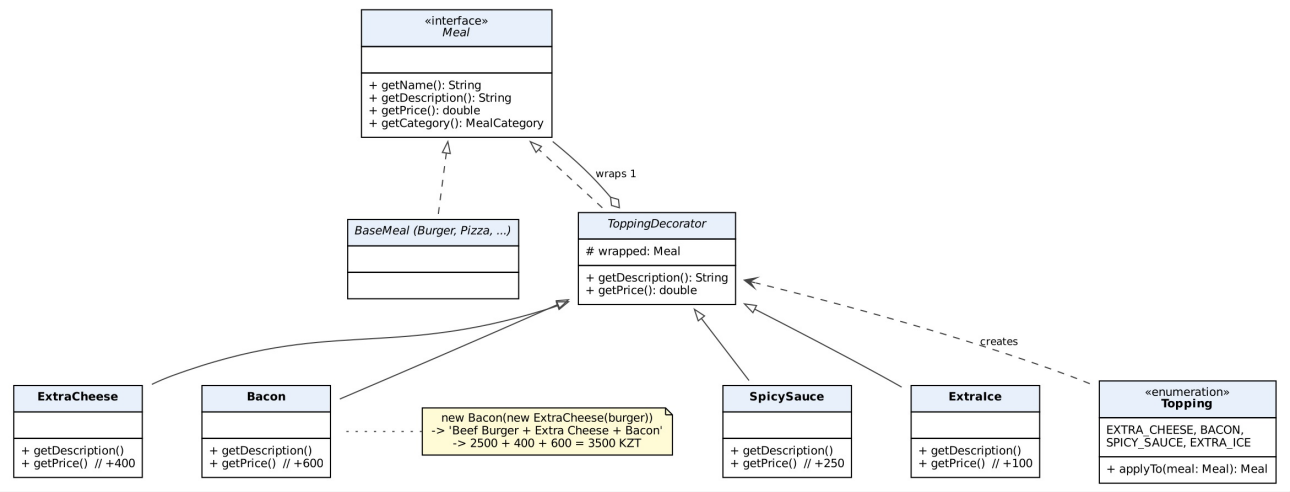


Figure 4. UML class diagram - Decorator

Key implementation.

decorator/ToppingDecorator.java

```java
package restaurant.decorator;

import restaurant.factory.Meal;
import restaurant.factory.MealCategory;

/**
 * DECORATOR pattern - abstract Decorator.
 *
 * Implements the same Meal interface as the wrapped object and forwards
 * every call to it. Concrete toppings add their own price and description
 * on top. Decorators can be stacked: Bacon(ExtraCheese(Burger)).
 *
 * Author: Yeskendir Abraimov (Student 1)
 */
public abstract class ToppingDecorator implements Meal {
    protected final Meal wrapped;

    protected ToppingDecorator(Meal wrapped) {
        this.wrapped = wrapped;
    }

    @Override
    public String getName() {
        return wrapped.getName();
    }

    @Override
    public String getDescription() {
        return wrapped.getDescription();
    }

    @Override
```

decorator/Bacon.java

```java
package restaurant.decorator;

import restaurant.factory.Meal;

/** DECORATOR - Concrete decorator (+600 KZT). Author: Yeskendir Abraimov (Student 1) */
public class Bacon extends ToppingDecorator {
    public Bacon(Meal wrapped) {
        super(wrapped);
    }

    @Override
    public String getDescription() {
        return wrapped.getDescription() + " + Bacon";
    }

    @Override
    public double getPrice() {
        return wrapped.getPrice() + 600;
    }
}
```

decorator/Topping.java (fragment)

```java
    /** Wraps the given meal with the matching concrete decorator. */
    public Meal applyTo(Meal meal) {
        switch (this) {
            case EXTRA_CHEESE: return new ExtraCheese(meal);
            case BACON:        return new Bacon(meal);
            case SPICY_SAUCE:  return new SpicySauce(meal);
            case EXTRA_ICE:    return new ExtraIce(meal);
            default: throw new IllegalStateException();
        }
    }
}
```

Interaction with other patterns: the decorators wrap objects produced by the **Factory Method**, and the decorated meals are later added to the order by the **Builder** and priced by the **Strategy** (Happy Hour checks the meal category, which the decorator forwards to the wrapped meal).

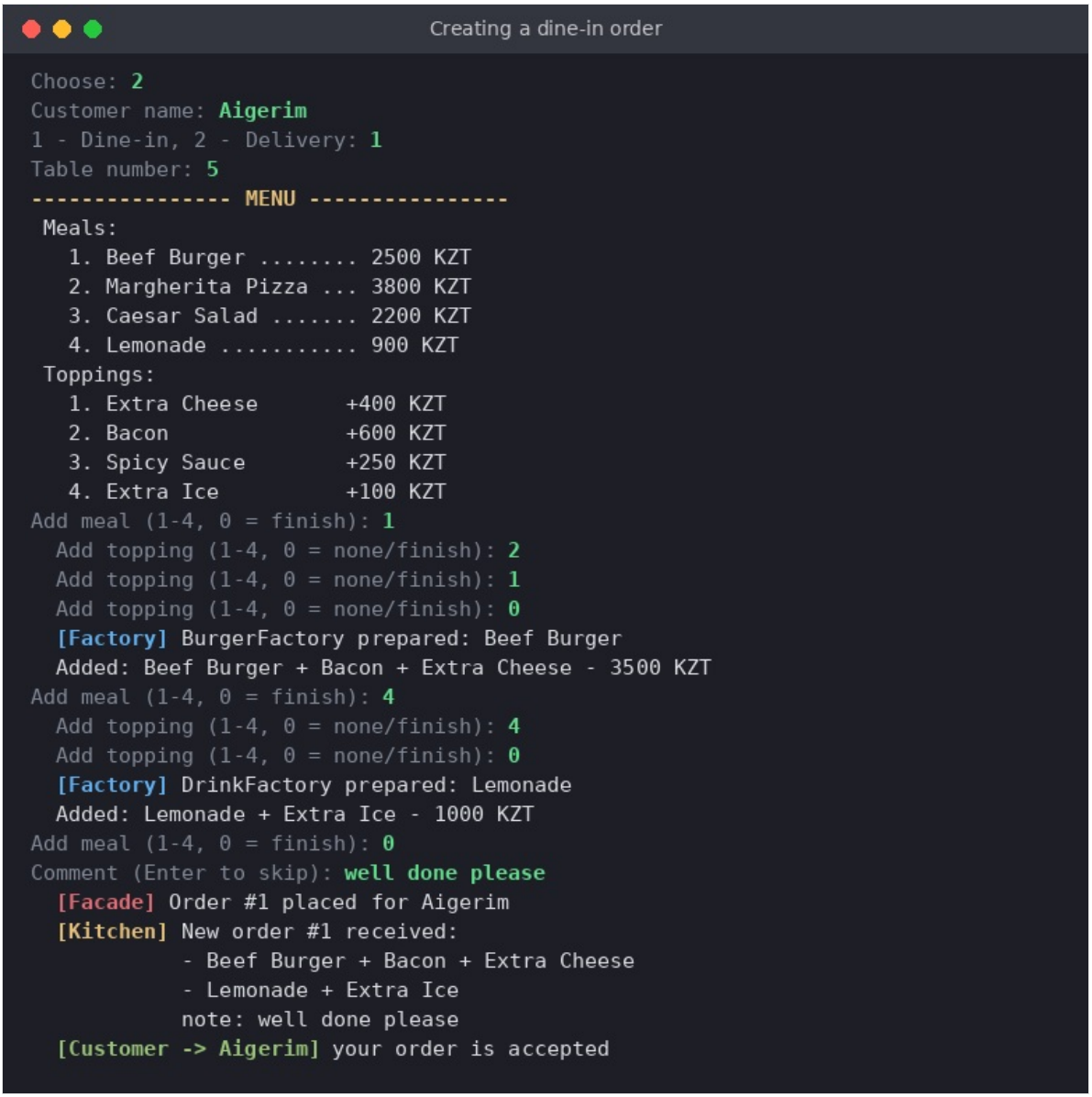


Figure 5. Decorator during interactive ordering - toppings change description and price of the burger and lemonade

2.3 Observer implemented by Dias Zhumakhan

Intent. Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Why it is used here. Every status change of an order (PLACED → COOKING → READY → SERVED → PAID, or CANCELLED) must reach several parties: the kitchen display, the customer and the cashier dashboard. The order does not know what these parties do - it only keeps a list of `OrderObserver`s and calls `update()` on each of them inside `setStatus()`. New listeners (e.g. an SMS service) can be added without changing `Order`.

Pattern role	Class in the project
Subject	`OrderSubject` (interface)
Concrete subject	`Order` - list of observers, `setStatus()` calls `notifyObservers()`
Observer	`OrderObserver` (interface) with `update(order, oldStatus, newStatus)`
Concrete observers	`KitchenDisplay`, `CustomerNotifier`, `CashierDashboard`

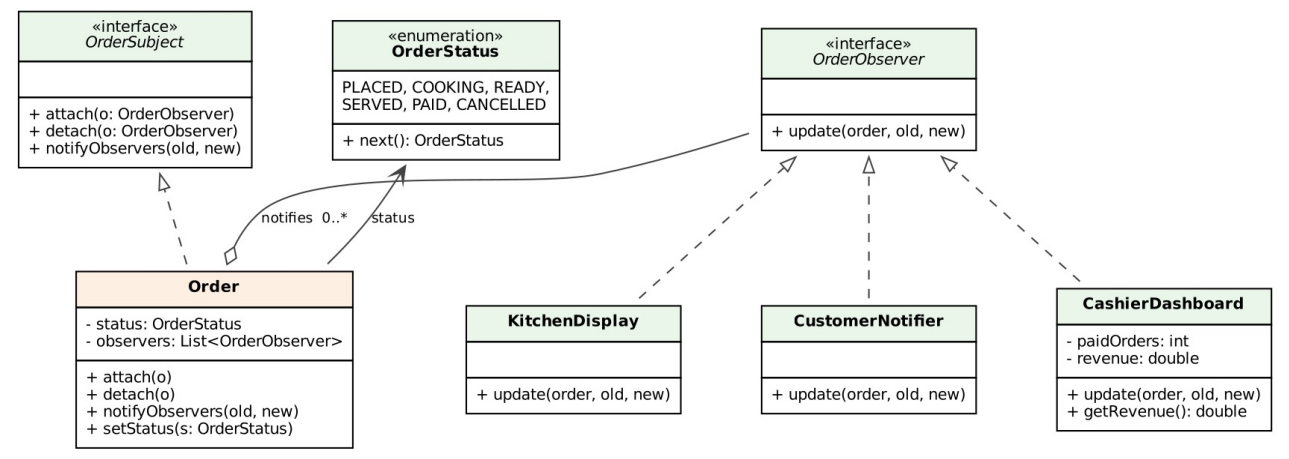


Figure 6. UML class diagram - Observer

Key implementation.

builder/Order.java (fragment)

```
    // ------------------------------------------------------------ OBSERVER (subject)
    @Override
    public void attach(OrderObserver observer) {
        observers.add(observer);
    }

    @Override
    public void detach(OrderObserver observer) {
        observers.remove(observer);
    }

    @Override
    public void notifyObservers(OrderStatus oldStatus, OrderStatus newStatus) {
        for (OrderObserver observer : observers) {
            observer.update(this, oldStatus, newStatus);
        }
    }

    /** Every status change automatically notifies all subscribed observers. */
    public void setStatus(OrderStatus newStatus) {
        OrderStatus old = this.status;
        this.status = newStatus;
        notifyObservers(old, newStatus);
    }
```

observer/OrderObserver.java

```java
package restaurant.observer;

import restaurant.builder.Order;

/**
 * OBSERVER pattern - Observer interface.
 * Every party interested in order changes (kitchen, customer, cashier)
 * implements this interface and is notified automatically.
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public interface OrderObserver {
    void update(Order order, OrderStatus oldStatus, OrderStatus newStatus);
}
```

observer/CashierDashboard.java

```java
package restaurant.observer;

import restaurant.builder.Order;

/**
 * OBSERVER pattern - Concrete observer: cashier dashboard.
 * Keeps live sales statistics; updated only when an order is paid.
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public class CashierDashboard implements OrderObserver {
    private int paidOrders;
    private double revenue;

    @Override
    public void update(Order order, OrderStatus oldStatus, OrderStatus newStatus) {
        if (newStatus == OrderStatus.PAID) {
            paidOrders++;
            revenue += order.getFinalTotal();
            System.out.printf("  [Cashier] Order #%d paid %.0f KZT via %s. Today: %d order(s), %.0f KZT%n",
                    order.getId(), order.getFinalTotal(), order.getPaymentMethod(), paidOrders, revenue);
        }
    }

    public int getPaidOrders() {
        return paidOrders;
    }

    public double getRevenue() {
        return revenue;
    }
}
```

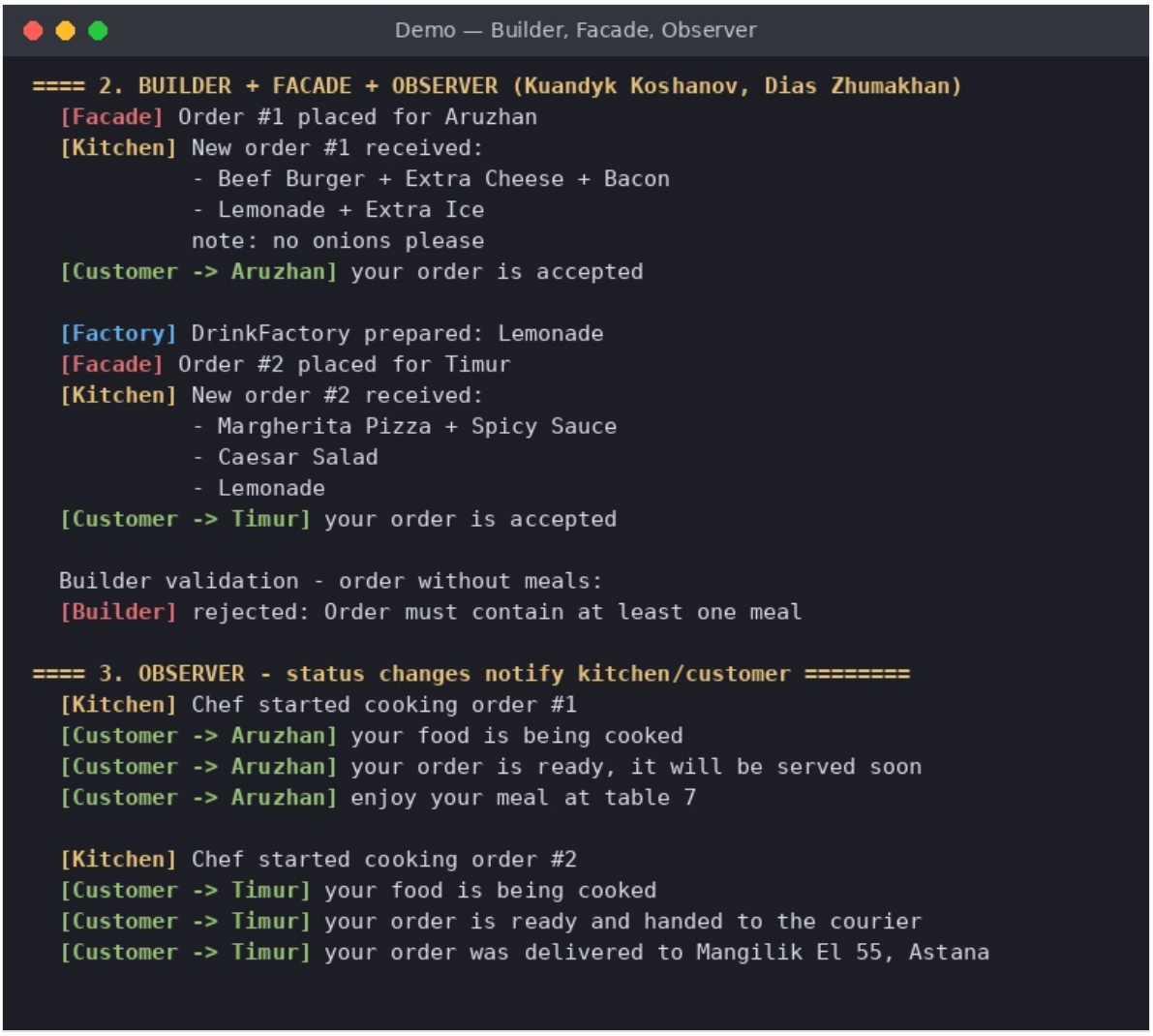


Figure 7. Observer - kitchen and customer are notified automatically when orders are placed

2.4 Strategy implemented by Dias Zhumakhan

Intent. Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

Why it is used here. The restaurant has several promotions (no discount, student -10%, Happy Hour -50% on drinks) and several payment methods (cash with change, bank card, Kaspi QR). Instead of a `switch` inside the order, each algorithm is a separate class behind a common interface. The order (context) holds a `DiscountStrategy` that can be replaced at run time, and the facade receives a `PaymentStrategy` at checkout. The demo calculates the same order with all three discount strategies to prove that they are interchangeable.

Pattern role	Class in the project
Strategy (discount)	`DiscountStrategy`
Concrete strategies	`NoDiscount`, `StudentDiscount`, `HappyHourDiscount`
Strategy (payment)	`PaymentStrategy`
Concrete strategies	`CashPayment`, `CardPayment`, `KaspiQrPayment`
Context	`Order` (discount), `RestaurantFacade.checkout()` (payment)

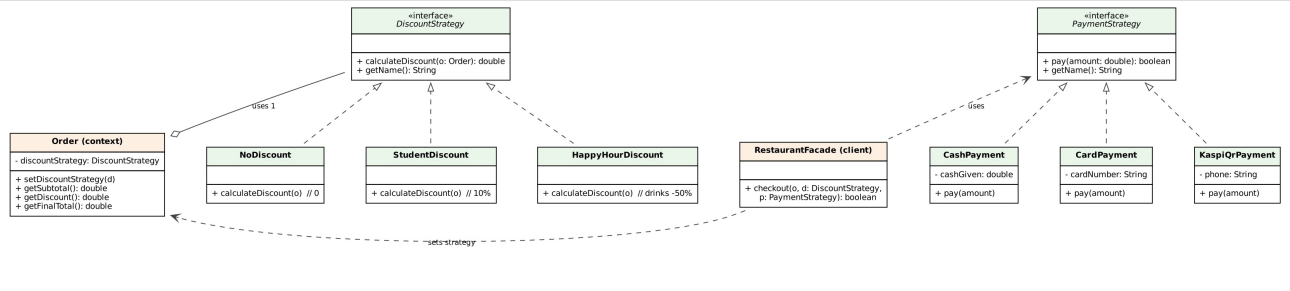


Figure 8. UML class diagram - Strategy

Key implementation.

`strategy/DiscountStrategy.java`

```
package restaurant.strategy;

import restaurant.builder.Order;

/**
 * STRATEGY pattern - strategy interface for discount algorithms.
 * The order total is calculated the same way; only the discount rule changes.
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public interface DiscountStrategy {
    /** @return discount amount in KZT (not the final price) */
    double calculateDiscount(Order order);

    String getName();
}
```

strategy/HappyHourDiscount.java

```java
package restaurant.strategy;

import restaurant.builder.Order;
import restaurant.factory.Meal;
import restaurant.factory.MealCategory;

/**
 * STRATEGY - Concrete strategy: 50% off all drinks.
 * Shows that a strategy can use details of the order, not just the total.
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public class HappyHourDiscount implements DiscountStrategy {
    @Override
    public double calculateDiscount(Order order) {
        double discount = 0;
        for (Meal meal : order.getItems()) {
            if (meal.getCategory() == MealCategory.DRINK) {
                discount += meal.getPrice() * 0.5;
            }
        }
        return discount;
    }

    @Override
    public String getName() {
        return "Happy Hour (drinks -50%)";
    }
}
```

builder/Order.java (fragment)

```java
    // ------------------------------------------------------------ STRATEGY (context)
    public void setDiscountStrategy(DiscountStrategy discountStrategy) {
        this.discountStrategy = discountStrategy;
    }

    public DiscountStrategy getDiscountStrategy() {
        return discountStrategy;
    }

    public double getSubtotal() {
        double sum = 0;
        for (Meal meal : items) {
            sum += meal.getPrice();
        }
        return sum;
    }

    public double getDiscount() {
        return discountStrategy.calculateDiscount(this);
    }

    public double getFinalTotal() {
        return getSubtotal() - getDiscount();
    }
```

strategy/CashPayment.java

```java
package restaurant.strategy;

/** STRATEGY - Concrete payment strategy: cash with change. Author: Dias Zhumakhan (Student 2) */
public class CashPayment implements PaymentStrategy {
    private final double cashGiven;

    public CashPayment(double cashGiven) {
        this.cashGiven = cashGiven;
    }

    @Override
    public boolean pay(double amount) {
        if (cashGiven < amount) {
            System.out.printf("  [Payment] Not enough cash: given %.0f, need %.0f KZT%n", cashGiven, amount);
            return false;
        }
        System.out.printf("  [Payment] Cash %.0f KZT received, change %.0f KZT%n", cashGiven, cashGiven - amount);
        return true;
    }

    @Override
    public String getName() {
        return "Cash";
    }
}
```

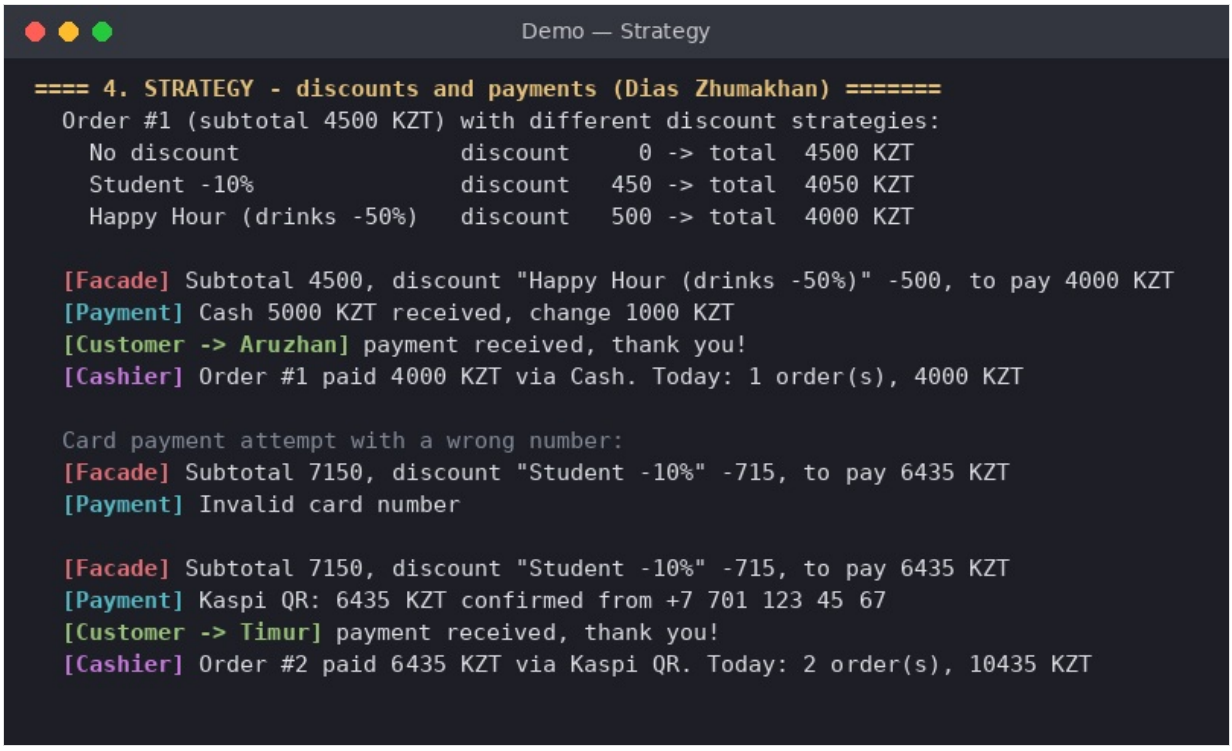


Figure 9. Strategy - the same order with three discount strategies, and three different payment strategies

2.5 Builder implemented by Kuandyk Koshanov

Intent. Separate the construction of a complex object from its representation, so that the same construction process can create different representations.

Why it is used here. An order has many parts and most of them are optional or mutually exclusive: a dine-in order needs a table number, a delivery order needs an address, a comment is optional, and the number of meals varies. A constructor with eight parameters would be unreadable and error-prone. `Order.Builder` collects the data step by step with a fluent interface and `build()` validates it before creating an immutable `Order` (the constructor of `Order` is private). The demo shows that an order without meals is rejected.

Pattern role	Class in the project
Builder	`Order.Builder` (static nested class)
Product	`Order` (private constructor, immutable list of items)
Director / client	`Main.createOrder()`, demo scenario; the built order is passed to `RestaurantFacade.placeOrder()`

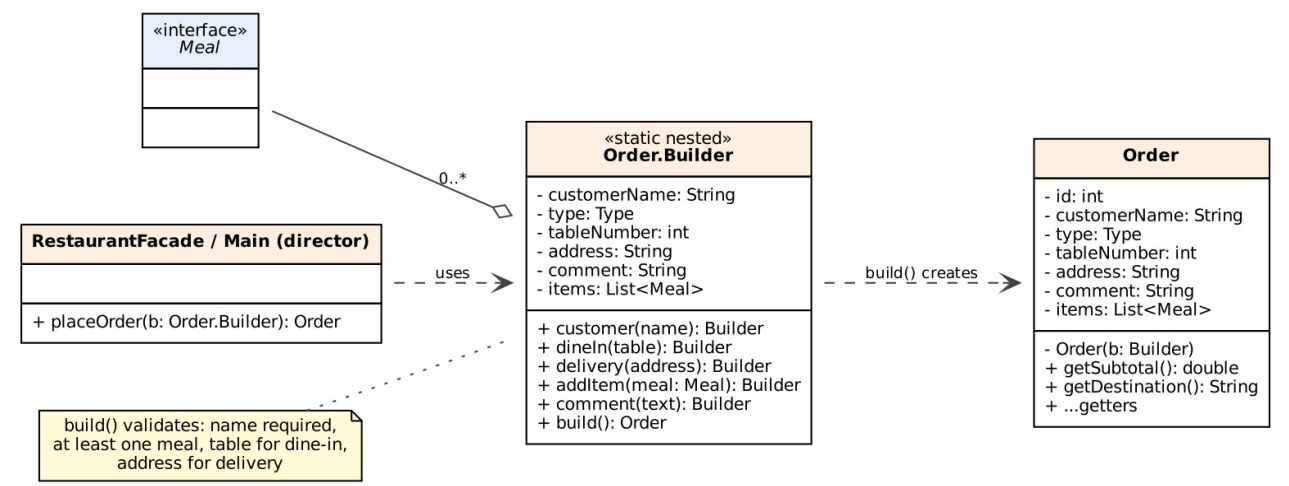


Figure 10. UML class diagram - Builder

Key implementation.

builder/Order.java (fragment)

```
    // ------------------------------------------------------------ BUILDER
    public static class Builder {
        private String customerName;
        private Type type = Type.DINE_IN;
        private int tableNumber;
        private String address = "";
        private String comment = "";
        private final List<Meal> items = new ArrayList<>();

        public Builder customer(String name) {
            this.customerName = name;
            return this;
        }

        public Builder dineIn(int tableNumber) {
            this.type = Type.DINE_IN;
            this.tableNumber = tableNumber;
            return this;
        }

        public Builder delivery(String address) {
            this.type = Type.DELIVERY;
            this.address = address;
            return this;
        }

        public Builder addItem(Meal meal) {
            this.items.add(meal);
            return this;
        }

        public Builder comment(String comment) {
            this.comment = comment;
            return this;
        }

        /** Validates the collected data and creates the immutable order. */
        public Order build() {
            if (customerName == null || customerName.trim().isEmpty()) {
                throw new IllegalStateException("Customer name is required");
            }
            if (items.isEmpty()) {
                throw new IllegalStateException("Order must contain at least one meal");
            }
            if (type == Type.DINE_IN && tableNumber <= 0) {
                throw new IllegalStateException("Dine-in order needs a table number");
            }
            if (type == Type.DELIVERY && address.trim().isEmpty()) {
                throw new IllegalStateException("Delivery order needs an address");
            }
            return new Order(this);
        }
    }
```

Main.java (fragment)

```
        Order dineIn = restaurant.placeOrder(new Order.Builder()
                .customer("Aruzhan")
                .dineIn(7)
                .addItem(burger)
                .addItem(lemonade)
                .comment("no onions please"));
```

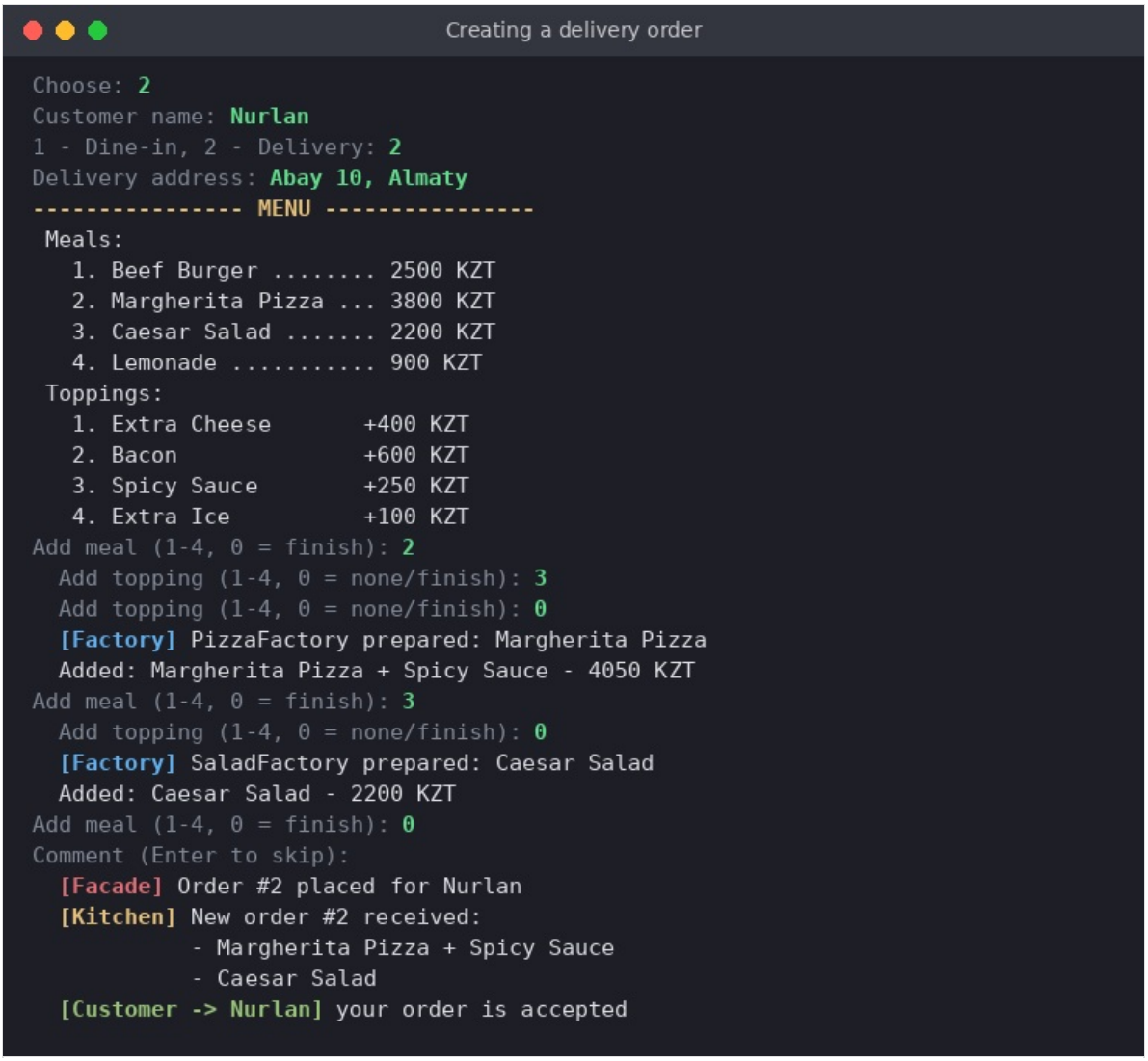


Figure 11. Builder - a delivery order is assembled step by step from the console

2.6 Bridge implemented by Kuandyk Koshanov

Intent. Decouple an abstraction from its implementation so that the two can vary independently.

Why it is used here. A receipt has two independent dimensions: *what* is printed (a short receipt with the total only, or a detailed receipt with every item, discount and payment method) and *how* it is printed (plain-text box for the console / thermal printer, or an HTML page that can be e-mailed). With inheritance we would need a class for every combination (ShortConsoleReceipt, ShortHtmlReceipt, DetailedConsoleReceipt, ...). With the Bridge, `Receipt` keeps a reference to a `ReceiptRenderer`, so 2 receipt types × 2 renderers give 4 combinations with only 4 classes, and adding a PDF renderer would not touch the receipt classes at all.

Pattern role	Class in the project
Abstraction	`Receipt` (field `renderer`)
Refined abstractions	`ShortReceipt`, `DetailedReceipt`
Implementor	`ReceiptRenderer`
Concrete implementors	`ConsoleRenderer`, `HtmlRenderer`

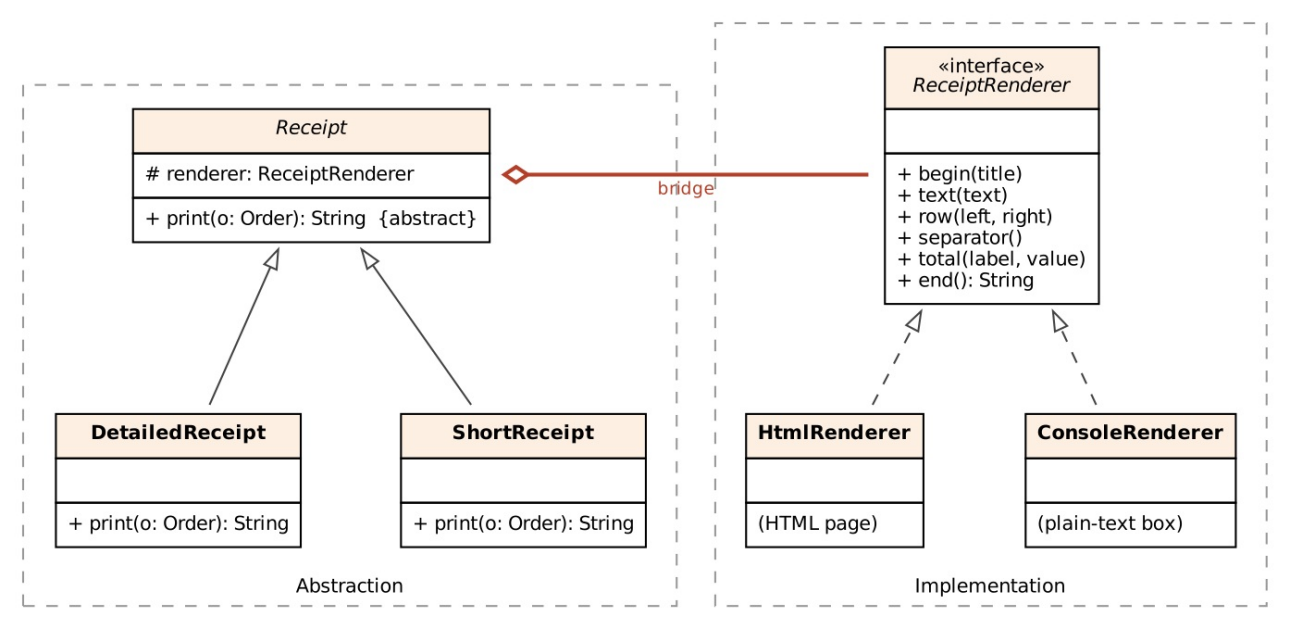


Figure 13. UML class diagram - Bridge

Key implementation.

bridge/Receipt.java

```java
package restaurant.bridge;

import restaurant.builder.Order;

/**
 * BRIDGE pattern - Abstraction.
 *
 * Holds a reference to a ReceiptRenderer (the "bridge"). Receipt types and
 * output formats vary independently: 2 receipt types x 2 renderers = 4
 * combinations without creating 4 subclasses.
 *
 * Author: Kuandyk Koshanov (Student 3)
 */
public abstract class Receipt {
    protected final ReceiptRenderer renderer;

    protected Receipt(ReceiptRenderer renderer) {
        this.renderer = renderer;
    }

    /** Builds the receipt for the given order and returns the rendered text. */
    public abstract String print(Order order);

    protected static String money(double value) {
        return String.format("%.0f KZT", value);
    }
}
```

bridge/ReceiptRenderer.java

```java
package restaurant.bridge;

/**
 * BRIDGE pattern - Implementor.
 *
 * Low-level formatting operations. The receipt (abstraction) only says
 * WHAT to print; the renderer decides HOW it looks (plain text, HTML, ...).
 *
 * Author: Kuandyk Koshanov (Student 3)
 */
public interface ReceiptRenderer {
    void begin(String title);

    void text(String text);

    void row(String left, String right);

    void separator();

    void total(String label, String value);

    String end();
}
```

bridge/ShortReceipt.java

```java
package restaurant.bridge;

import restaurant.builder.Order;

/**
 * BRIDGE pattern - Refined abstraction: compact receipt (total only).
 * Author: Kuandyk Koshanov (Student 3)
 */
public class ShortReceipt extends Receipt {

    public ShortReceipt(ReceiptRenderer renderer) {
        super(renderer);
    }

    @Override
    public String print(Order order) {
        renderer.begin("RECEIPT #" + order.getId());
        renderer.row(order.getCustomerName(), order.getItems().size() + " item(s)");
        renderer.total("Total", money(order.getFinalTotal()));
        return renderer.end();
    }
}
```

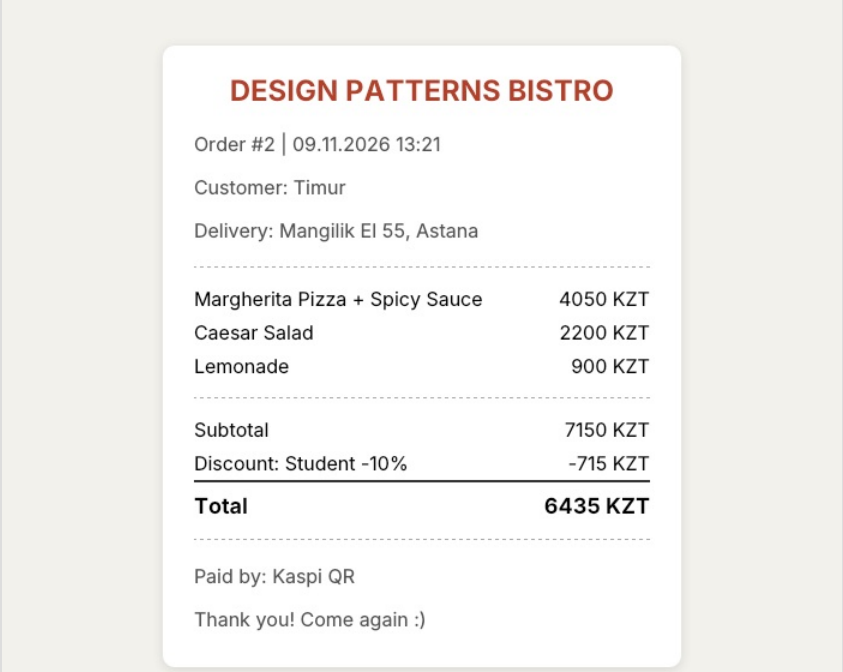


Figure 12. Bridge - DetailedReceipt + HtmlRenderer opened in a web browser

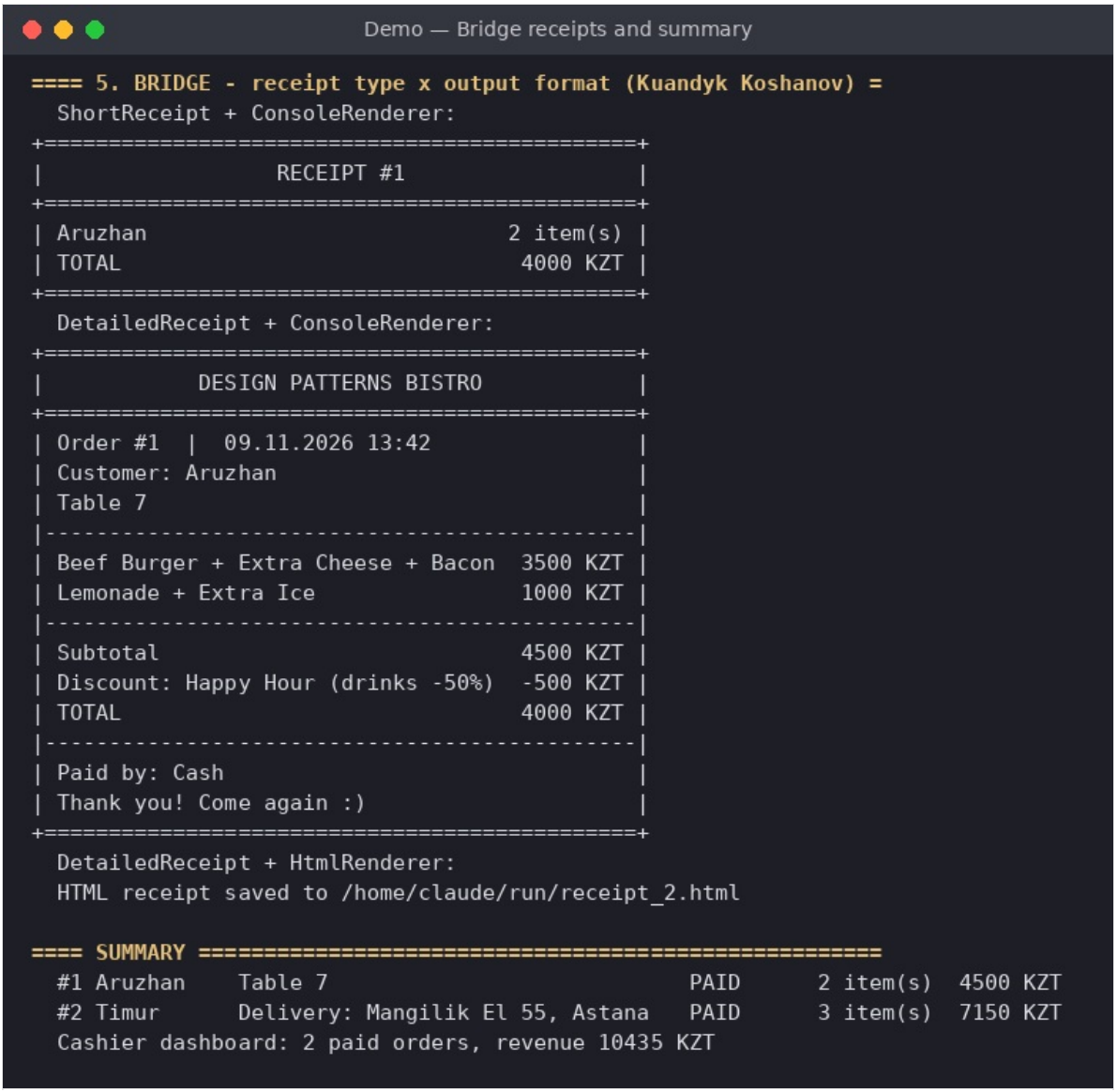


Figure 14. Bridge - the same order printed as a short and a detailed console receipt; a detailed HTML receipt is saved to a file

2.7 Facade implemented by Kuandyk Koshanov

Intent. Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.

Why it is used here. Placing and paying for an order involves classes from all other packages: factories, decorators, the builder, three observers, discount and payment strategies and the receipt bridge. Without a facade the console client would depend on all of them and repeat the same coordination code in many places. `RestaurantFacade` hides this complexity behind a handful of methods - `orderMeal()`, `placeOrder()`, `nextStatus()`, `cancel()`, `checkout()`, `printReceipt()` - and also guarantees that every order is subscribed to all observers and that a paid or cancelled order cannot be paid again.

Pattern role	Class in the project
Facade	`RestaurantFacade`
Subsystem classes	`MealFactory`, `Topping`/decorators, `Order.Builder`, `Order`, observers, strategies, `Receipt`/renderers
Client	`Main` - uses only the facade

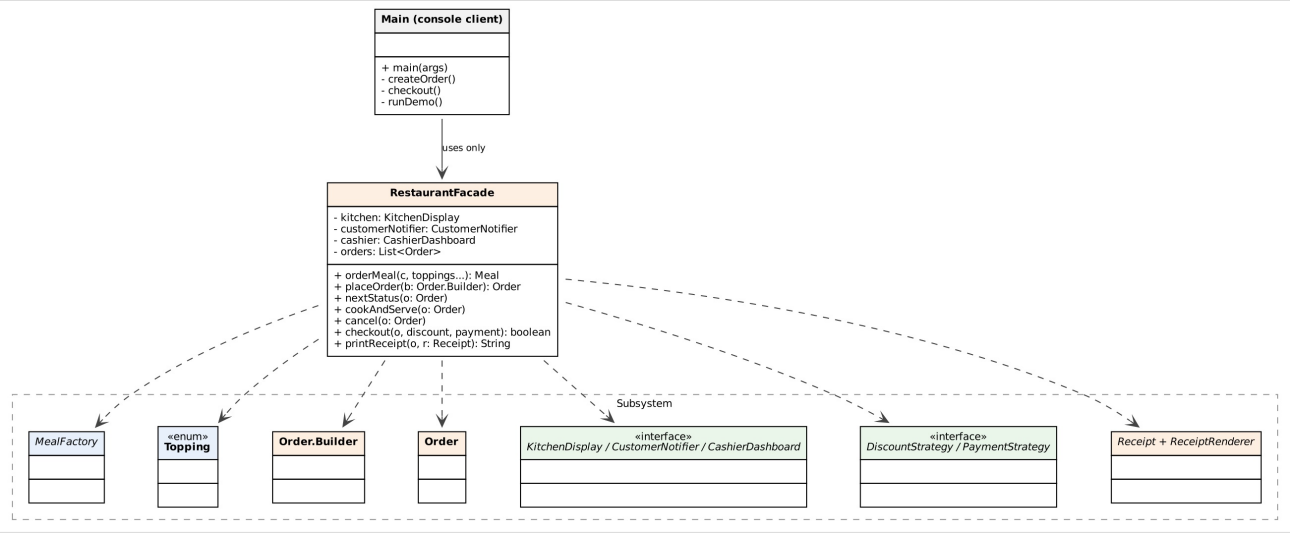


Figure 15. UML class diagram - Facade

Key implementation.

facade/RestaurantFacade.java (fragment)

```java
public class RestaurantFacade {
    private final KitchenDisplay kitchen = new KitchenDisplay();
    private final CustomerNotifier customerNotifier = new CustomerNotifier();
    private final CashierDashboard cashier = new CashierDashboard();
    private final List<Order> orders = new ArrayList<>();

    /** Factory Method + Decorator: creates a meal and wraps it with toppings. */
    public Meal orderMeal(MealCategory category, Topping... toppings) {
        Meal meal = MealFactory.forCategory(category).prepareMeal();
        for (Topping topping : toppings) {
            meal = topping.applyTo(meal);
        }
        return meal;
    }

    /** Builder (built by the caller) + Observer: subscribes all observers and places the order. */
    public Order placeOrder(Order.Builder builder) {
        Order order = builder.build();
        order.attach(kitchen);
        order.attach(customerNotifier);
        order.attach(cashier);
        orders.add(order);
        System.out.println("  [Facade] Order #" + order.getId() + " placed for " + order.getCustomerName());
        order.setStatus(OrderStatus.PLACED);
        return order;
    }

    /** Moves the order to the next kitchen status; observers are notified. */
    public void nextStatus(Order order) {
        OrderStatus current = order.getStatus();
        if (current == OrderStatus.SERVED || current == OrderStatus.PAID || current == OrderStatus.CANCELLED) {
            System.out.println("  [Facade] Order #" + order.getId() + " is already " + current);
            return;
        }
        order.setStatus(current.next());
    }

    /** Runs the full kitchen cycle: PLACED -> COOKING -> READY -> SERVED. */
    public void cookAndServe(Order order) {
        while (order.getStatus() != OrderStatus.SERVED
                && order.getStatus() != OrderStatus.PAID
                && order.getStatus() != OrderStatus.CANCELLED) {
            nextStatus(order);
        }
    }

    public void cancel(Order order) {
        if (order.getStatus() == OrderStatus.PAID) {
            System.out.println("  [Facade] Paid order cannot be cancelled");
            return;
        }
        order.setStatus(OrderStatus.CANCELLED);
    }

    /** Strategy: applies the chosen discount and pays with the chosen method. */
    public boolean checkout(Order order, DiscountStrategy discount, PaymentStrategy payment) {
        if (order.getStatus() == OrderStatus.PAID) {
            System.out.println("  [Facade] Order #" + order.getId() + " is already paid");
            return false;
        }
        if (order.getStatus() == OrderStatus.CANCELLED) {
            System.out.println("  [Facade] Order #" + order.getId() + " is cancelled");
            return false;
        }
        order.setDiscountStrategy(discount);
        System.out.printf("  [Facade] Subtotal %.0f, discount \"%s\" -%.0f, to pay %.0f KZT%n",
                order.getSubtotal(), discount.getName(), order.getDiscount(), order.getFinalTotal());
        if (!payment.pay(order.getFinalTotal())) {
            return false;
        }
        order.setPaymentMethod(payment.getName());
        order.setStatus(OrderStatus.PAID);
        return true;
    }

    /** Bridge: any receipt type with any renderer. */
    public String printReceipt(Order order, Receipt receipt) {
        return receipt.print(order);
    }
```

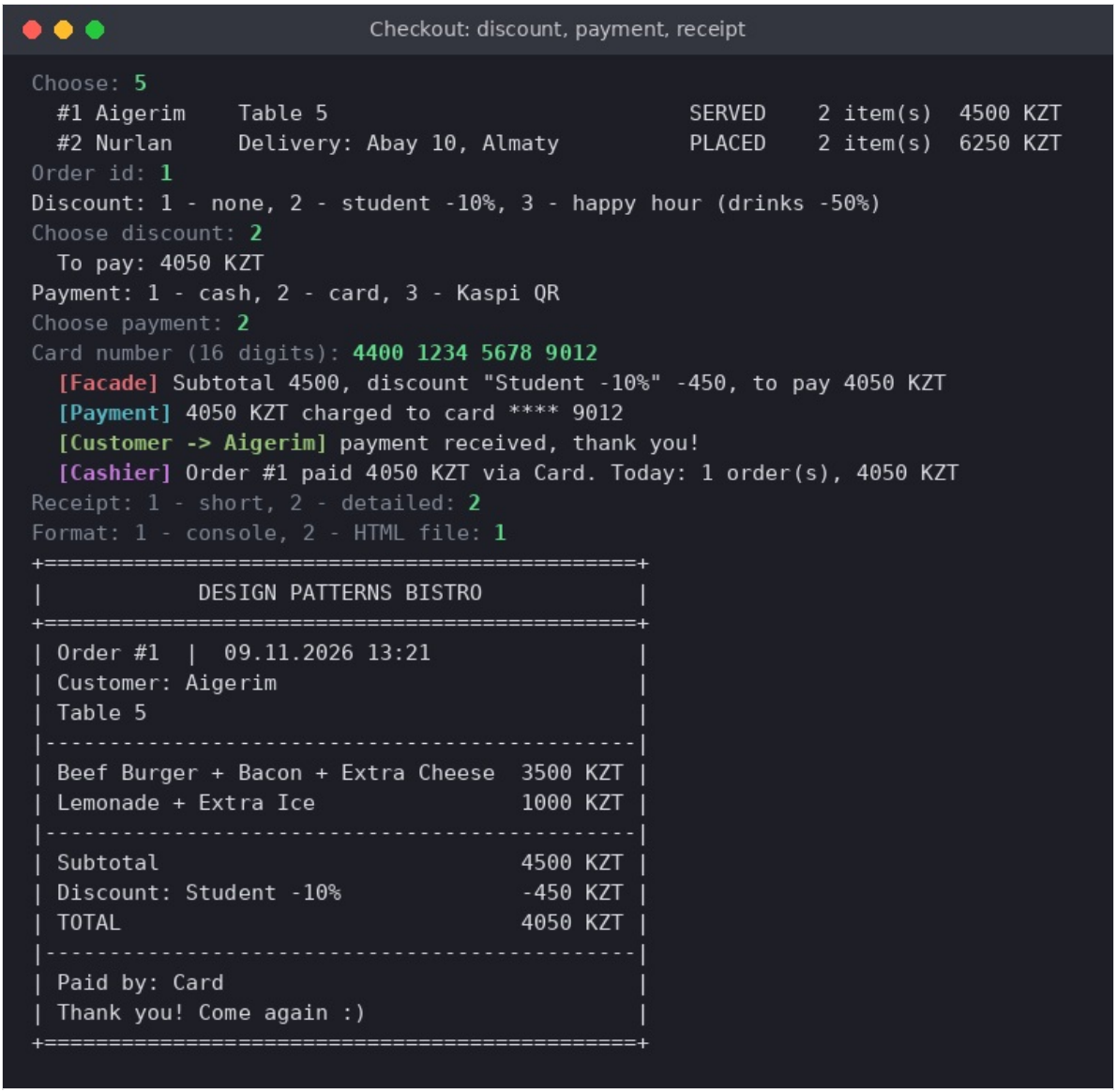


Figure 16. Facade - one checkout() call applies the discount, takes the payment, notifies observers; then a receipt is printed

2.8 How the patterns work together

The requirement is that the patterns work in a coordinated manner inside one project, not as isolated examples. The table below lists the direct interactions between them; Figure 17 shows the whole flow of one order as a UML sequence diagram.

From	To	Interaction
Facade	Factory Method	`orderMeal()` selects a concrete factory with `MealFactory.forCategory()` and calls `prepareMeal()`.
Factory Method	Decorator	Meals created by factories are the concrete components wrapped by topping decorators.
Decorator	Builder	Decorated meals are added to the order with `Order.Builder.addItem()`.
Builder	Observer	The built `Order` is the subject; the facade attaches the kitchen, customer and cashier observers.
Strategy	Builder / Decorator	`Order` is the context of `DiscountStrategy`; `HappyHourDiscount` reads the category of decorated meals.
Strategy	Observer	A successful `PaymentStrategy.pay()` changes the status to PAID, which updates `CashierDashboard` statistics.
Bridge	Builder / Strategy	Receipts read the order items, the discount strategy name and the payment method.
Facade	all	Coordinates the full scenario; the client depends only on the facade.

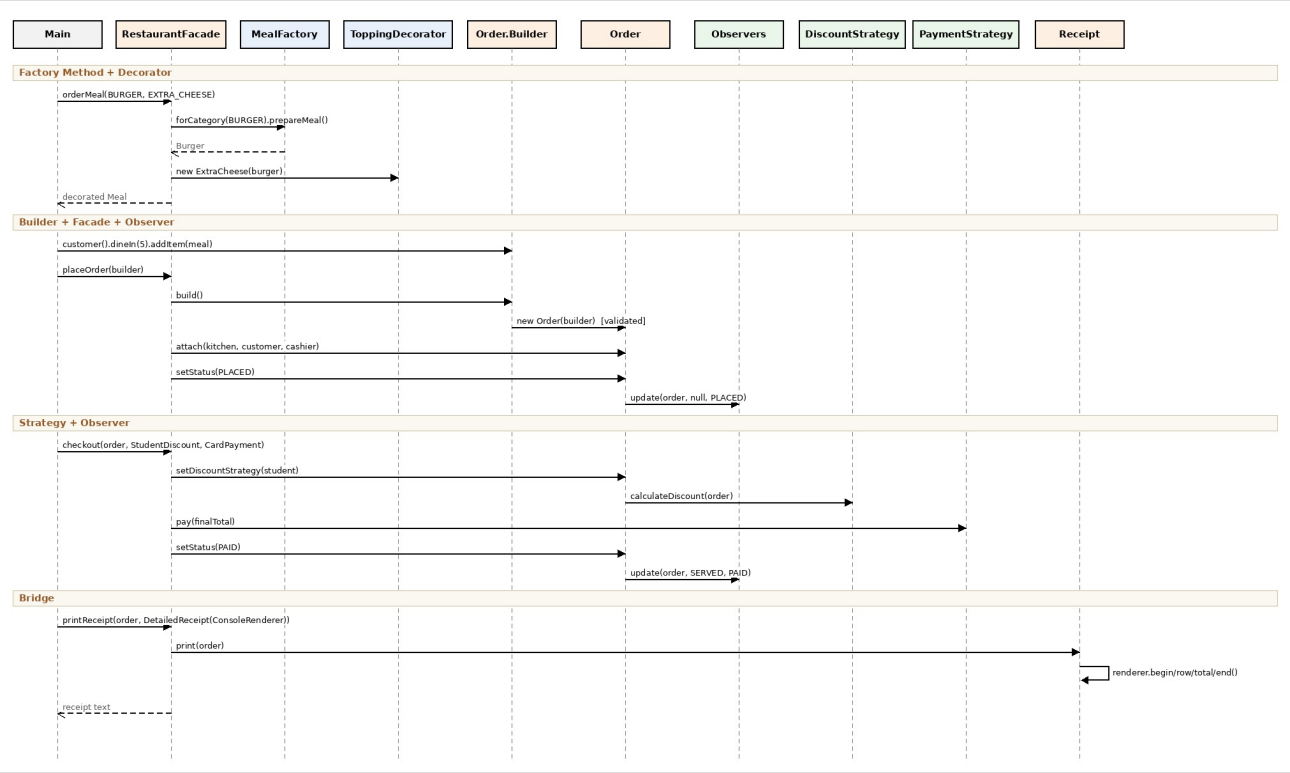


Figure 17. UML sequence diagram - creating, placing, paying for and printing an order

2.9 Application screenshots

The application has an interactive console menu. The screenshots below are taken from one continuous session: the user views the menu, creates a dine-in and a delivery order, moves the first order through the kitchen statuses, pays for it and finally cancels the second order.

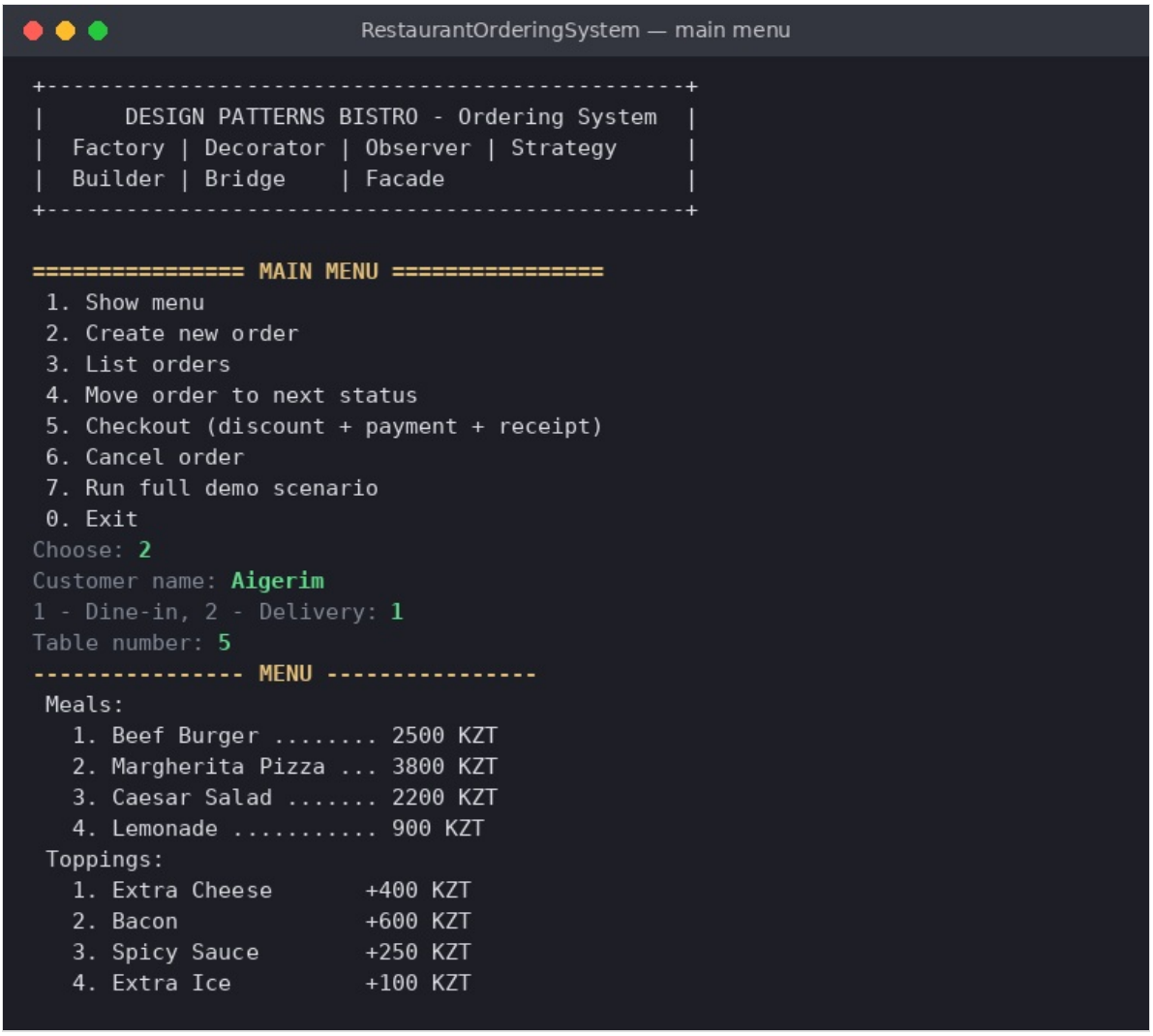


Figure 18. Main menu and the restaurant menu with meals and toppings

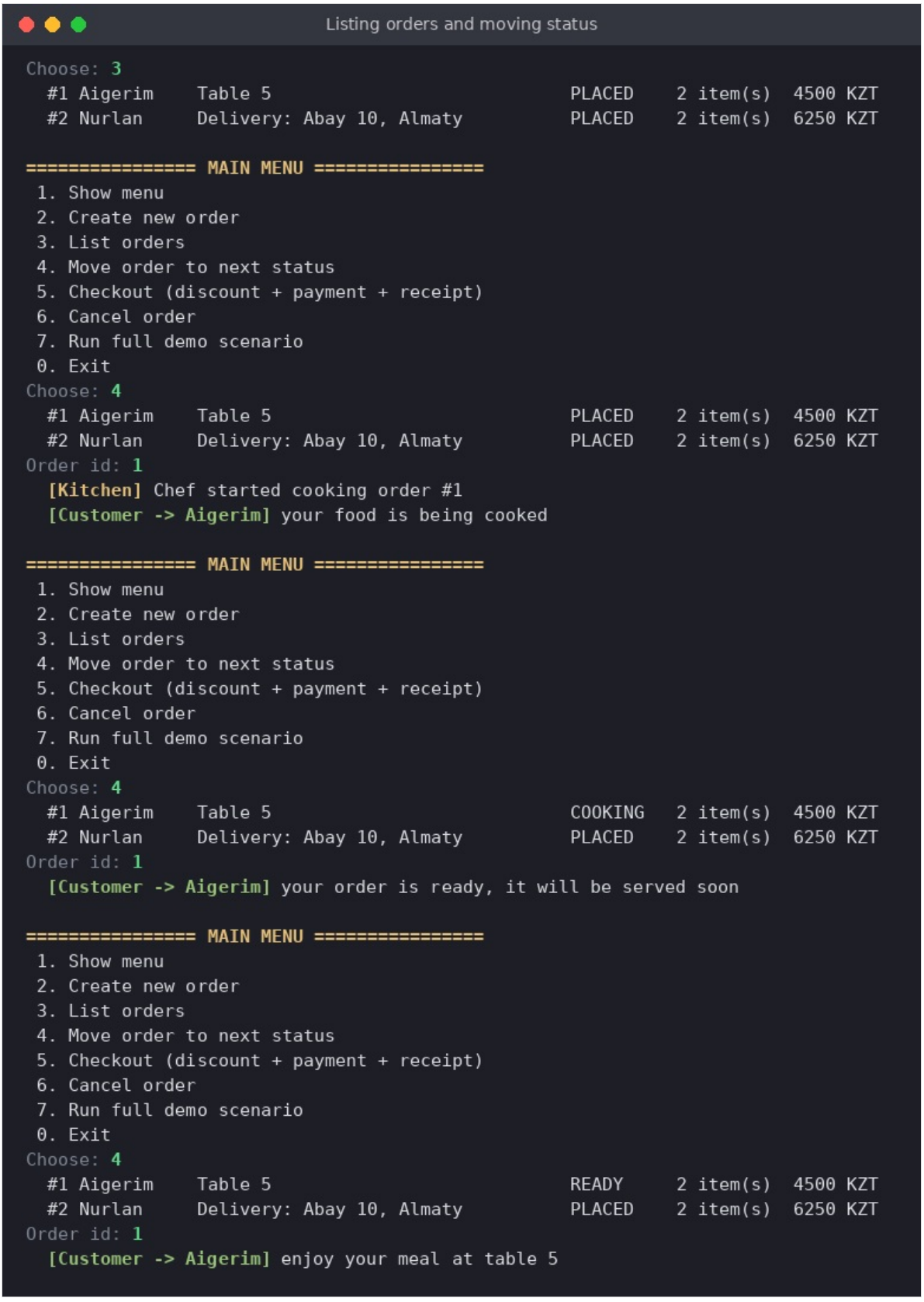


Figure 19. Listing orders and moving order #1 through COOKING, READY and SERVED (Observer notifications)

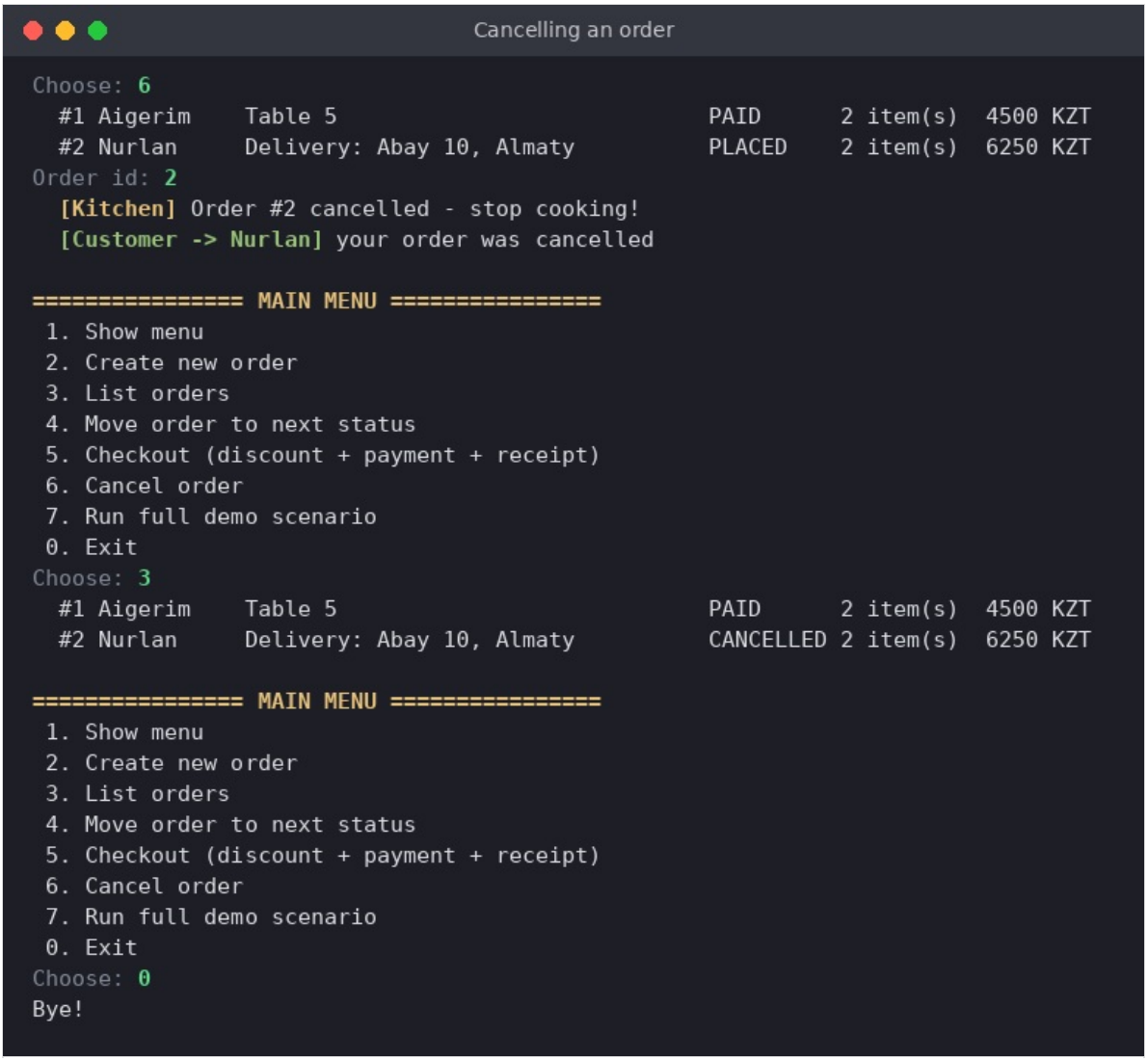


Figure 20. Cancelling order #2 - the kitchen and the customer are notified; final list of orders

2.10 Building and running

The project requires only a JDK (version 11 or newer). From the project root:

```
# Linux / macOS                       # Windows
./build.sh                            build.bat
./run.sh           (interactive)      run.bat
./run.sh --demo    (scripted demo)    run.bat --demo

# or manually
javac -d out $(find src -name "*.java")
java -cp out restaurant.Main --demo
```

The project can also be opened directly in IntelliJ IDEA or Eclipse: mark `src` as the sources root and run `restaurant.Main`.

3. Conclusion

In this project our team designed and implemented a Restaurant Ordering System in Java that integrates **seven design patterns** - Factory Method, Decorator, Observer, Strategy, Builder, Bridge and Facade - into a single working application. Every team member implemented at least two patterns: Yeskendir Abraimov - Factory Method and Decorator, Dias Zhumakhan - Observer and Strategy, Kuandyk Koshanov - Builder, Bridge and Facade. All patterns are fully operational, which is demonstrated both in the interactive mode and in the scripted `--demo` scenario.

The most important result is that the patterns do not live in isolation. Objects flow through the system from one pattern to the next: a meal is created by a factory, decorated with toppings, collected by the builder into an order, the order notifies its observers, its price is calculated by an interchangeable strategy, and the result is printed through the bridge - all orchestrated by the facade. Because of this design:

- new meals, toppings, discounts, payment methods, observers or receipt formats can be added by writing **new classes** rather than modifying existing ones (Open/Closed principle);
- each class has a single, clear responsibility, and each package belongs to one pattern and one author;
- the client code is very small because it depends only on the facade.

Working on the project helped us understand not only how each pattern is structured, but also *when* it should be used and how patterns complement each other in a real application.

4. Further work

- **Graphical or web interface.** Replace the console client with a JavaFX desktop UI or a Spring Boot REST API. Because the client depends only on `RestaurantFacade`, the rest of the system would not change.
- **Persistent storage.** Save menu items and orders in a database (e.g. PostgreSQL via JDBC/JPA) instead of memory.
- **Real notifications.** Add new observers that send SMS, e-mail or Telegram messages to customers and a live dashboard for managers.
- **Integration with delivery services and payment providers.** Real APIs (Wolt, Glovo, Kaspi Pay) have different interfaces and could be connected through the Adapter pattern.
- **More receipt formats.** Add a `PdfRenderer` or a fiscal-printer renderer to the Bridge without changing the receipt classes.
- **Different cuisines.** Use the Abstract Factory pattern to create families of related meals (Italian, Kazakh, Japanese menus).
- **Reports.** Apply the Visitor pattern to calculate statistics (calories, taxes, sales per category) over orders and meals.
- **Automated tests.** Cover every pattern with JUnit 5 unit tests and set up continuous integration.

References

1. E. Gamma, R. Helm, R. Johnson, J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1994.
2. E. Freeman, E. Robson. *Head First Design Patterns*, 2nd ed. O'Reilly Media, 2020.
3. R. C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship.* Prentice Hall, 2008.
4. A. Shvets. *Dive Into Design Patterns.* Refactoring.Guru, https://refactoring.guru/design-patterns
5. java-design-patterns project, https://github.com/iluwatar/java-design-patterns
6. Graphviz - graph visualization software, https://graphviz.org

Appendix A. Full source code

Complete source code of the project (also provided in the attached ZIP archive).

A.1 Factory Method (Yeskendir Abraimov)

factory/Meal.java

```java
package restaurant.factory;

/**
 * Product interface of the Factory Method pattern.
 * It is also the Component interface of the Decorator pattern,
 * so toppings can wrap any meal created by a factory.
 *
 * Author: Yeskendir Abraimov (Student 1)
 */
public interface Meal {
    String getName();

    String getDescription();

    double getPrice();

    MealCategory getCategory();
}
```

factory/MealCategory.java

```java
package restaurant.factory;

/** Kinds of meals the restaurant can produce. Author: Yeskendir Abraimov (Student 1) */
public enum MealCategory {
    BURGER, PIZZA, SALAD, DRINK
}
```

factory/BaseMeal.java

```java
package restaurant.factory;

/**
 * Common state for every concrete meal (concrete products).
 *
 * Author: Yeskendir Abraimov (Student 1)
 */
public abstract class BaseMeal implements Meal {
    private final String name;
    private final String description;
    private final double price;
    private final MealCategory category;

    protected BaseMeal(String name, String description, double price, MealCategory category) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.category = category;
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public String getDescription() {
        return description;
    }

    @Override
    public double getPrice() {
        return price;
    }

    @Override
    public MealCategory getCategory() {
        return category;
    }

    @Override
    public String toString() {
        return getDescription() + String.format(" - %.0f KZT", getPrice());
    }
}
```

factory/Burger.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete product. Author: Yeskendir Abraimov (Student 1) */
class Burger extends BaseMeal {
    Burger() {
        super("Burger", "Beef Burger", 2500, MealCategory.BURGER);
    }
}
```

factory/Pizza.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete product. Author: Yeskendir Abraimov (Student 1) */
class Pizza extends BaseMeal {
    Pizza() {
        super("Pizza", "Margherita Pizza", 3800, MealCategory.PIZZA);
    }
}
```

factory/Salad.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete product. Author: Yeskendir Abraimov (Student 1) */
class Salad extends BaseMeal {
    Salad() {
        super("Salad", "Caesar Salad", 2200, MealCategory.SALAD);
    }
}
```

factory/Drink.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete product. Author: Yeskendir Abraimov (Student 1) */
class Drink extends BaseMeal {
    Drink() {
        super("Drink", "Lemonade", 900, MealCategory.DRINK);
    }
}
```

factory/MealFactory.java

```java
package restaurant.factory;

/**
 * FACTORY METHOD pattern - Creator.
 *
 * createMeal() is the factory method: each subclass decides which concrete
 * Meal to instantiate. prepareMeal() is the common workflow that every
 * kitchen station follows and that relies on the factory method.
 *
 * Author: Yeskendir Abraimov (Student 1)
 */
public abstract class MealFactory {

    /** The factory method - implemented by concrete creators. */
    protected abstract Meal createMeal();

    /** Common business logic that uses the product created by the subclass. */
    public Meal prepareMeal() {
        Meal meal = createMeal();
        System.out.println("  [Factory] " + getClass().getSimpleName()
                + " prepared: " + meal.getDescription());
        return meal;
    }

    /** Returns the right concrete creator for the requested category. */
    public static MealFactory forCategory(MealCategory category) {
        switch (category) {
            case BURGER: return new BurgerFactory();
            case PIZZA:  return new PizzaFactory();
            case SALAD:  return new SaladFactory();
            case DRINK:  return new DrinkFactory();
            default: throw new IllegalArgumentException("Unknown meal: " + category);
        }
    }
}
```

factory/BurgerFactory.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete creator. Author: Yeskendir Abraimov (Student 1) */
public class BurgerFactory extends MealFactory {
    @Override
    protected Meal createMeal() {
        return new Burger();
    }
}
```

factory/PizzaFactory.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete creator. Author: Yeskendir Abraimov (Student 1) */
public class PizzaFactory extends MealFactory {
    @Override
    protected Meal createMeal() {
        return new Pizza();
    }
}
```

factory/SaladFactory.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete creator. Author: Yeskendir Abraimov (Student 1) */
public class SaladFactory extends MealFactory {
    @Override
    protected Meal createMeal() {
        return new Salad();
    }
}
```

factory/DrinkFactory.java

```java
package restaurant.factory;

/** FACTORY METHOD - Concrete creator. Author: Yeskendir Abraimov (Student 1) */
public class DrinkFactory extends MealFactory {
    @Override
    protected Meal createMeal() {
        return new Drink();
    }
}
```

A.2 Decorator (Yeskendir Abraimov)

decorator/ToppingDecorator.java

```java
package restaurant.decorator;

import restaurant.factory.Meal;
import restaurant.factory.MealCategory;

/**
 * DECORATOR pattern - abstract Decorator.
 *
 * Implements the same Meal interface as the wrapped object and forwards
 * every call to it. Concrete toppings add their own price and description
 * on top. Decorators can be stacked: Bacon(ExtraCheese(Burger)).
 *
 * Author: Yeskendir Abraimov (Student 1)
 */
public abstract class ToppingDecorator implements Meal {
    protected final Meal wrapped;

    protected ToppingDecorator(Meal wrapped) {
        this.wrapped = wrapped;
    }

    @Override
    public String getName() {
        return wrapped.getName();
    }

    @Override
    public String getDescription() {
        return wrapped.getDescription();
    }

    @Override
    public double getPrice() {
        return wrapped.getPrice();
    }

    @Override
    public MealCategory getCategory() {
        return wrapped.getCategory();
    }

    @Override
    public String toString() {
        return getDescription() + String.format(" - %.0f KZT", getPrice());
    }
}
```

decorator/ExtraCheese.java

```java
package restaurant.decorator;

import restaurant.factory.Meal;

/** DECORATOR - Concrete decorator (+400 KZT). Author: Yeskendir Abraimov (Student 1) */
public class ExtraCheese extends ToppingDecorator {
    public ExtraCheese(Meal wrapped) {
        super(wrapped);
    }

    @Override
    public String getDescription() {
        return wrapped.getDescription() + " + Extra Cheese";
    }

    @Override
    public double getPrice() {
        return wrapped.getPrice() + 400;
    }
}
```

decorator/Bacon.java

```java
package restaurant.decorator;

import restaurant.factory.Meal;

/** DECORATOR - Concrete decorator (+600 KZT). Author: Yeskendir Abraimov (Student 1) */
public class Bacon extends ToppingDecorator {
    public Bacon(Meal wrapped) {
        super(wrapped);
    }

    @Override
    public String getDescription() {
        return wrapped.getDescription() + " + Bacon";
    }

    @Override
    public double getPrice() {
        return wrapped.getPrice() + 600;
    }
}
```

decorator/SpicySauce.java

```java
package restaurant.decorator;

import restaurant.factory.Meal;

/** DECORATOR - Concrete decorator (+250 KZT). Author: Yeskendir Abraimov (Student 1) */
public class SpicySauce extends ToppingDecorator {
    public SpicySauce(Meal wrapped) {
        super(wrapped);
    }

    @Override
    public String getDescription() {
        return wrapped.getDescription() + " + Spicy Sauce";
    }

    @Override
    public double getPrice() {
        return wrapped.getPrice() + 250;
    }
}
```

decorator/ExtraIce.java

```java
package restaurant.decorator;

import restaurant.factory.Meal;

/** DECORATOR - Concrete decorator (+100 KZT, for drinks). Author: Yeskendir Abraimov (Student 1) */
public class ExtraIce extends ToppingDecorator {
    public ExtraIce(Meal wrapped) {
        super(wrapped);
    }

    @Override
    public String getDescription() {
        return wrapped.getDescription() + " + Extra Ice";
    }

    @Override
    public double getPrice() {
        return wrapped.getPrice() + 100;
    }
}
```

decorator/Topping.java

```java
package restaurant.decorator;

import restaurant.factory.Meal;

/**
 * Helper enum used by the menu/facade to apply a decorator by name.
 * Author: Yeskendir Abraimov (Student 1)
 */
public enum Topping {
    EXTRA_CHEESE("Extra Cheese", 400),
    BACON("Bacon", 600),
    SPICY_SAUCE("Spicy Sauce", 250),
    EXTRA_ICE("Extra Ice", 100);

    private final String label;
    private final double price;

    Topping(String label, double price) {
        this.label = label;
        this.price = price;
    }

    public String getLabel() {
        return label;
    }

    public double getPrice() {
        return price;
    }

    /** Wraps the given meal with the matching concrete decorator. */
    public Meal applyTo(Meal meal) {
        switch (this) {
            case EXTRA_CHEESE: return new ExtraCheese(meal);
            case BACON:        return new Bacon(meal);
            case SPICY_SAUCE:  return new SpicySauce(meal);
            case EXTRA_ICE:    return new ExtraIce(meal);
            default: throw new IllegalStateException();
        }
    }
}
```

A.3 Observer (Dias Zhumakhan)

observer/OrderStatus.java

```java
package restaurant.observer;

/** Life cycle of an order. Author: Dias Zhumakhan (Student 2) */
public enum OrderStatus {
    PLACED, COOKING, READY, SERVED, PAID, CANCELLED;

    /** Next status in the normal kitchen flow (PLACED -> COOKING -> READY -> SERVED). */
    public OrderStatus next() {
        switch (this) {
            case PLACED:  return COOKING;
            case COOKING: return READY;
            case READY:   return SERVED;
            default:      return this;
        }
    }
}
```

observer/OrderSubject.java

```java
package restaurant.observer;

/**
 * OBSERVER pattern - Subject interface (implemented by Order).
 * Author: Dias Zhumakhan (Student 2)
 */
public interface OrderSubject {
    void attach(OrderObserver observer);

    void detach(OrderObserver observer);

    void notifyObservers(OrderStatus oldStatus, OrderStatus newStatus);
}
```

observer/OrderObserver.java

```java
package restaurant.observer;

import restaurant.builder.Order;

/**
 * OBSERVER pattern - Observer interface.
 * Every party interested in order changes (kitchen, customer, cashier)
 * implements this interface and is notified automatically.
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public interface OrderObserver {
    void update(Order order, OrderStatus oldStatus, OrderStatus newStatus);
}
```

observer/KitchenDisplay.java

```java
package restaurant.observer;

import restaurant.builder.Order;
import restaurant.factory.Meal;

/**
 * OBSERVER pattern - Concrete observer: screen in the kitchen.
 * Reacts to new and cancelled orders.
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public class KitchenDisplay implements OrderObserver {

    @Override
    public void update(Order order, OrderStatus oldStatus, OrderStatus newStatus) {
        if (newStatus == OrderStatus.PLACED) {
            System.out.println("  [Kitchen] New order #" + order.getId() + " received:");
            for (Meal meal : order.getItems()) {
                System.out.println("            - " + meal.getDescription());
            }
            if (!order.getComment().isEmpty()) {
                System.out.println("            note: " + order.getComment());
            }
        } else if (newStatus == OrderStatus.COOKING) {
            System.out.println("  [Kitchen] Chef started cooking order #" + order.getId());
        } else if (newStatus == OrderStatus.CANCELLED) {
            System.out.println("  [Kitchen] Order #" + order.getId() + " cancelled - stop cooking!");
        }
    }
}
```

observer/CustomerNotifier.java

```java
package restaurant.observer;

import restaurant.builder.Order;

/**
 * OBSERVER pattern - Concrete observer: notifications sent to the customer
 * (in a real system this would be an SMS / push message).
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public class CustomerNotifier implements OrderObserver {

    @Override
    public void update(Order order, OrderStatus oldStatus, OrderStatus newStatus) {
        String message;
        switch (newStatus) {
            case PLACED:    message = "your order is accepted"; break;
            case COOKING:   message = "your food is being cooked"; break;
            case READY:     message = order.isDelivery()
                                    ? "your order is ready and handed to the courier"
                                    : "your order is ready, it will be served soon"; break;
            case SERVED:    message = order.isDelivery()
                                    ? "your order was delivered to " + order.getAddress()
                                    : "enjoy your meal at table " + order.getTableNumber(); break;
            case PAID:      message = "payment received, thank you!"; break;
            case CANCELLED: message = "your order was cancelled"; break;
            default:        message = newStatus.name();
        }
        System.out.println("  [Customer -> " + order.getCustomerName() + "] " + message);
    }
}
```

observer/CashierDashboard.java

```java
package restaurant.observer;

import restaurant.builder.Order;

/**
 * OBSERVER pattern - Concrete observer: cashier dashboard.
 * Keeps live sales statistics; updated only when an order is paid.
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public class CashierDashboard implements OrderObserver {
    private int paidOrders;
    private double revenue;

    @Override
    public void update(Order order, OrderStatus oldStatus, OrderStatus newStatus) {
        if (newStatus == OrderStatus.PAID) {
            paidOrders++;
            revenue += order.getFinalTotal();
            System.out.printf("  [Cashier] Order #%d paid %.0f KZT via %s. Today: %d order(s), %.0f KZT%n",
                    order.getId(), order.getFinalTotal(), order.getPaymentMethod(), paidOrders, revenue);
        }
    }

    public int getPaidOrders() {
        return paidOrders;
    }

    public double getRevenue() {
        return revenue;
    }
}
```

A.4 Strategy (Dias Zhumakhan)

strategy/DiscountStrategy.java

```java
package restaurant.strategy;

import restaurant.builder.Order;

/**
 * STRATEGY pattern - strategy interface for discount algorithms.
 * The order total is calculated the same way; only the discount rule changes.
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public interface DiscountStrategy {
    /** @return discount amount in KZT (not the final price) */
    double calculateDiscount(Order order);

    String getName();
}
```

strategy/NoDiscount.java

```java
package restaurant.strategy;

import restaurant.builder.Order;

/** STRATEGY - Concrete strategy: regular price. Author: Dias Zhumakhan (Student 2) */
public class NoDiscount implements DiscountStrategy {
    @Override
    public double calculateDiscount(Order order) {
        return 0;
    }

    @Override
    public String getName() {
        return "No discount";
    }
}
```

strategy/StudentDiscount.java

```java
package restaurant.strategy;

import restaurant.builder.Order;

/** STRATEGY - Concrete strategy: 10% off the whole order. Author: Dias Zhumakhan (Student 2) */
public class StudentDiscount implements DiscountStrategy {
    private static final double RATE = 0.10;

    @Override
    public double calculateDiscount(Order order) {
        return order.getSubtotal() * RATE;
    }

    @Override
    public String getName() {
        return "Student -10%";
    }
}
```

strategy/HappyHourDiscount.java

```java
package restaurant.strategy;

import restaurant.builder.Order;
import restaurant.factory.Meal;
import restaurant.factory.MealCategory;

/**
 * STRATEGY - Concrete strategy: 50% off all drinks.
 * Shows that a strategy can use details of the order, not just the total.
 *
 * Author: Dias Zhumakhan (Student 2)
 */
public class HappyHourDiscount implements DiscountStrategy {
    @Override
    public double calculateDiscount(Order order) {
        double discount = 0;
        for (Meal meal : order.getItems()) {
            if (meal.getCategory() == MealCategory.DRINK) {
                discount += meal.getPrice() * 0.5;
            }
        }
        return discount;
    }

    @Override
    public String getName() {
        return "Happy Hour (drinks -50%)";
    }
}
```

strategy/PaymentStrategy.java

```java
package restaurant.strategy;

/**
 * STRATEGY pattern - strategy interface for payment methods.
 * Author: Dias Zhumakhan (Student 2)
 */
public interface PaymentStrategy {
    /** @return true if the payment succeeded */
    boolean pay(double amount);

    String getName();
}
```

strategy/CashPayment.java

```java
package restaurant.strategy;

/** STRATEGY - Concrete payment strategy: cash with change. Author: Dias Zhumakhan (Student 2) */
public class CashPayment implements PaymentStrategy {
    private final double cashGiven;

    public CashPayment(double cashGiven) {
        this.cashGiven = cashGiven;
    }

    @Override
    public boolean pay(double amount) {
        if (cashGiven < amount) {
            System.out.printf("  [Payment] Not enough cash: given %.0f, need %.0f KZT%n", cashGiven, amount);
            return false;
        }
        System.out.printf("  [Payment] Cash %.0f KZT received, change %.0f KZT%n", cashGiven, cashGiven - amount);
        return true;
    }

    @Override
    public String getName() {
        return "Cash";
    }
}
```

strategy/CardPayment.java

```java
package restaurant.strategy;

/** STRATEGY - Concrete payment strategy: bank card. Author: Dias Zhumakhan (Student 2) */
public class CardPayment implements PaymentStrategy {
    private final String cardNumber;

    public CardPayment(String cardNumber) {
        this.cardNumber = cardNumber.replaceAll("\\s", "");
    }

    @Override
    public boolean pay(double amount) {
        if (cardNumber.length() != 16 || !cardNumber.matches("\\d+")) {
            System.out.println("  [Payment] Invalid card number");
            return false;
        }
        System.out.printf("  [Payment] %.0f KZT charged to card **** %s%n", amount, cardNumber.substring(12));
        return true;
    }

    @Override
    public String getName() {
        return "Card";
    }
}
```

strategy/KaspiQrPayment.java

```java
package restaurant.strategy;

/** STRATEGY - Concrete payment strategy: Kaspi QR by phone number. Author: Dias Zhumakhan (Student 2) */
public class KaspiQrPayment implements PaymentStrategy {
    private final String phone;

    public KaspiQrPayment(String phone) {
        this.phone = phone;
    }

    @Override
    public boolean pay(double amount) {
        System.out.printf("  [Payment] Kaspi QR: %.0f KZT confirmed from %s%n", amount, phone);
        return true;
    }

    @Override
    public String getName() {
        return "Kaspi QR";
    }
}
```

A.5 Builder (Kuandyk Koshanov)

builder/Order.java

```java
package restaurant.builder;

import restaurant.factory.Meal;
import restaurant.observer.OrderObserver;
```

```java
import restaurant.observer.OrderStatus;
import restaurant.observer.OrderSubject;
import restaurant.strategy.DiscountStrategy;
import restaurant.strategy.NoDiscount;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * BUILDER pattern - Product built by Order.Builder (author: Kuandyk Koshanov, Student 3).
 *
 * An order has many optional parts (table OR address, comment, several meals),
 * so instead of a constructor with 8 parameters it is assembled step by step
 * by the nested static Builder, which also validates the result.
 *
 * The Order is also the SUBJECT of the OBSERVER pattern (author: Dias Zhumakhan, Student 2)
 * and the CONTEXT of the STRATEGY pattern (discount) - this is where the
 * patterns of different students work together.
 */
public class Order implements OrderSubject {

    public enum Type { DINE_IN, DELIVERY }

    private static int nextId = 1;

    private final int id;
    private final String customerName;
    private final Type type;
    private final int tableNumber;
    private final String address;
    private final String comment;
    private final List<Meal> items;
    private final LocalDateTime createdAt;

    private OrderStatus status;
    private final List<OrderObserver> observers = new ArrayList<>();

    private DiscountStrategy discountStrategy = new NoDiscount();
    private String paymentMethod = "-";

    /** Private: an Order can be created only through the Builder. */
    private Order(Builder builder) {
        this.id = nextId++;
        this.customerName = builder.customerName;
        this.type = builder.type;
        this.tableNumber = builder.tableNumber;
        this.address = builder.address;
        this.comment = builder.comment;
        this.items = Collections.unmodifiableList(new ArrayList<>(builder.items));
        this.createdAt = LocalDateTime.now();
    }

    // ------------------------------------------------------------ BUILDER
    public static class Builder {
        private String customerName;
        private Type type = Type.DINE_IN;
        private int tableNumber;
        private String address = "";
        private String comment = "";
        private final List<Meal> items = new ArrayList<>();

        public Builder customer(String name) {
            this.customerName = name;
            return this;
        }

        public Builder dineIn(int tableNumber) {
            this.type = Type.DINE_IN;
            this.tableNumber = tableNumber;
            return this;
        }

        public Builder delivery(String address) {
            this.type = Type.DELIVERY;
            this.address = address;
            return this;
        }

        public Builder addItem(Meal meal) {
            this.items.add(meal);
            return this;
        }

        public Builder comment(String comment) {
            this.comment = comment;
            return this;
        }

        /** Validates the collected data and creates the immutable order. */
        public Order build() {
```

```
            if (customerName == null || customerName.trim().isEmpty()) {
                throw new IllegalStateException("Customer name is required");
            }
            if (items.isEmpty()) {
                throw new IllegalStateException("Order must contain at least one meal");
            }
            if (type == Type.DINE_IN && tableNumber <= 0) {
                throw new IllegalStateException("Dine-in order needs a table number");
            }
            if (type == Type.DELIVERY && address.trim().isEmpty()) {
                throw new IllegalStateException("Delivery order needs an address");
            }
            return new Order(this);
        }
    }

    // ------------------------------------------------------------ OBSERVER (subject)
    @Override
    public void attach(OrderObserver observer) {
        observers.add(observer);
    }

    @Override
    public void detach(OrderObserver observer) {
        observers.remove(observer);
    }

    @Override
    public void notifyObservers(OrderStatus oldStatus, OrderStatus newStatus) {
        for (OrderObserver observer : observers) {
            observer.update(this, oldStatus, newStatus);
        }
    }

    /** Every status change automatically notifies all subscribed observers. */
    public void setStatus(OrderStatus newStatus) {
        OrderStatus old = this.status;
        this.status = newStatus;
        notifyObservers(old, newStatus);
    }

    // ------------------------------------------------------------ STRATEGY (context)
    public void setDiscountStrategy(DiscountStrategy discountStrategy) {
        this.discountStrategy = discountStrategy;
    }

    public DiscountStrategy getDiscountStrategy() {
        return discountStrategy;
    }

    public double getSubtotal() {
        double sum = 0;
        for (Meal meal : items) {
            sum += meal.getPrice();
        }
        return sum;
    }

    public double getDiscount() {
        return discountStrategy.calculateDiscount(this);
    }

    public double getFinalTotal() {
        return getSubtotal() - getDiscount();
    }

    // ------------------------------------------------------------ getters
    public int getId() { return id; }
    public String getCustomerName() { return customerName; }
    public Type getType() { return type; }
    public boolean isDelivery() { return type == Type.DELIVERY; }
    public int getTableNumber() { return tableNumber; }
    public String getAddress() { return address; }
    public String getComment() { return comment; }
    public List<Meal> getItems() { return items; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public OrderStatus getStatus() { return status; }
    public String getPaymentMethod() { return paymentMethod; }
    public void setPaymentMethod(String paymentMethod) { this.paymentMethod = paymentMethod; }

    public String getDestination() {
        return isDelivery() ? "Delivery: " + address : "Table " + tableNumber;
    }

    @Override
    public String toString() {
        return String.format("#%d %-10s %-34s %-9s %d item(s)  %.0f KZT",
                id, customerName, getDestination(), status, items.size(), getSubtotal());
    }
}
```

A.6 Bridge (Kuandyk Koshanov)

bridge/ReceiptRenderer.java

```java
package restaurant.bridge;

/**
 * BRIDGE pattern - Implementor.
 *
 * Low-level formatting operations. The receipt (abstraction) only says
 * WHAT to print; the renderer decides HOW it looks (plain text, HTML, ...).
 *
 * Author: Kuandyk Koshanov (Student 3)
 */
public interface ReceiptRenderer {
    void begin(String title);

    void text(String text);

    void row(String left, String right);

    void separator();

    void total(String label, String value);

    String end();
}
```

bridge/ConsoleRenderer.java

```java
package restaurant.bridge;

/**
 * BRIDGE pattern - Concrete implementor: plain-text receipt for the console
 * or a thermal printer.
 *
 * Author: Kuandyk Koshanov (Student 3)
 */
public class ConsoleRenderer implements ReceiptRenderer {
    private static final int WIDTH = 46;
    private StringBuilder out;

    @Override
    public void begin(String title) {
        out = new StringBuilder();
        out.append('+').append("=".repeat(WIDTH)).append("+\n");
        int pad = (WIDTH - title.length()) / 2;
        out.append('|').append(" ".repeat(pad)).append(title)
           .append(" ".repeat(WIDTH - pad - title.length())).append("|\n");
        out.append('+').append("=".repeat(WIDTH)).append("+\n");
    }

    @Override
    public void text(String text) {
        out.append("| ").append(fit(text, WIDTH - 2)).append(" |\n");
    }

    @Override
    public void row(String left, String right) {
        int space = WIDTH - 2 - right.length();
        out.append("| ").append(fit(left, space)).append(right).append(" |\n");
    }

    @Override
    public void separator() {
        out.append('|').append("-".repeat(WIDTH)).append("|\n");
    }

    @Override
    public void total(String label, String value) {
        row(label.toUpperCase(), value);
    }

    @Override
    public String end() {
        out.append('+').append("=".repeat(WIDTH)).append("+\n");
        return out.toString();
    }

    private static String fit(String s, int width) {
        if (s.length() > width) {
            return s.substring(0, width - 3) + "...";
        }
        return s + " ".repeat(width - s.length());
    }
}
```

bridge/HtmlRenderer.java

```
package restaurant.bridge;

/**
 * BRIDGE pattern - Concrete implementor: HTML receipt that can be e-mailed
 * to the customer or opened in a browser.
 *
 * Author: Kuandyk Koshanov (Student 3)
 */
public class HtmlRenderer implements ReceiptRenderer {
    private StringBuilder out;

    @Override
    public void begin(String title) {
        out = new StringBuilder();
        out.append("<!DOCTYPE html>\n<html><head><meta charset=\"UTF-8\"><title>").append(escape(title))
           .append("</title>\n<style>")
           .append("body{font-family:Arial,sans-serif;background:#f3f1ec;padding:30px}")
           .append(".r{background:#fff;width:380px;margin:auto;padding:22px 26px;border-radius:10px;")
           .append("box-shadow:0 2px 10px rgba(0,0,0,.12)}")
           .append("h2{text-align:center;margin:0 0 12px;color:#b5402a}")
           .append("table{width:100%;border-collapse:collapse}td{padding:4px 0}")
           .append("td.v{text-align:right}tr.t td{font-weight:bold;font-size:18px;border-top:2px solid #333;padding-
top:8px}")
           .append("p{margin:4px 0;color:#555}hr{border:0;border-top:1px dashed #aaa}")
           .append("</style></head><body><div class=\"r\">\n<h2>").append(escape(title)).append("</h2>\n<table>\n");
    }

    @Override
    public void text(String text) {
        out.append("<tr><td colspan=\"2\"><p>").append(escape(text)).append("</p></td></tr>\n");
    }

    @Override
    public void row(String left, String right) {
        out.append("<tr><td>").append(escape(left)).append("</td><td class=\"v\">")
           .append(escape(right)).append("</td></tr>\n");
    }

    @Override
    public void separator() {
        out.append("<tr><td colspan=\"2\"><hr></td></tr>\n");
    }

    @Override
    public void total(String label, String value) {
        out.append("<tr class=\"t\"><td>").append(escape(label)).append("</td><td class=\"v\">")
           .append(escape(value)).append("</td></tr>\n");
    }

    @Override
    public String end() {
        out.append("</table>\n</div></body></html>\n");
        return out.toString();
    }

    private static String escape(String s) {
        return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;");
    }
}
```

bridge/Receipt.java

```
package restaurant.bridge;

import restaurant.builder.Order;

/**
 * BRIDGE pattern - Abstraction.
 *
 * Holds a reference to a ReceiptRenderer (the "bridge"). Receipt types and
 * output formats vary independently: 2 receipt types x 2 renderers = 4
 * combinations without creating 4 subclasses.
 *
 * Author: Kuandyk Koshanov (Student 3)
 */
public abstract class Receipt {
    protected final ReceiptRenderer renderer;

    protected Receipt(ReceiptRenderer renderer) {
        this.renderer = renderer;
    }

    /** Builds the receipt for the given order and returns the rendered text. */
    public abstract String print(Order order);

    protected static String money(double value) {
        return String.format("%.0f KZT", value);
    }
}
```

bridge/ShortReceipt.java

```java
package restaurant.bridge;

import restaurant.builder.Order;

/**
 * BRIDGE pattern - Refined abstraction: compact receipt (total only).
 * Author: Kuandyk Koshanov (Student 3)
 */
public class ShortReceipt extends Receipt {

    public ShortReceipt(ReceiptRenderer renderer) {
        super(renderer);
    }

    @Override
    public String print(Order order) {
        renderer.begin("RECEIPT #" + order.getId());
        renderer.row(order.getCustomerName(), order.getItems().size() + " item(s)");
        renderer.total("Total", money(order.getFinalTotal()));
        return renderer.end();
    }
}
```

bridge/DetailedReceipt.java

```java
package restaurant.bridge;

import restaurant.builder.Order;
import restaurant.factory.Meal;

import java.time.format.DateTimeFormatter;

/**
 * BRIDGE pattern - Refined abstraction: full receipt with every item,
 * discount and payment method.
 *
 * Author: Kuandyk Koshanov (Student 3)
 */
public class DetailedReceipt extends Receipt {
    private static final DateTimeFormatter FMT = DateTimeFormatter.ofPattern("dd.MM.yyyy HH:mm");

    public DetailedReceipt(ReceiptRenderer renderer) {
        super(renderer);
    }

    @Override
    public String print(Order order) {
        renderer.begin("DESIGN PATTERNS BISTRO");
        renderer.text("Order #" + order.getId() + "  |  " + order.getCreatedAt().format(FMT));
        renderer.text("Customer: " + order.getCustomerName());
        renderer.text(order.getDestination());
        renderer.separator();
        for (Meal meal : order.getItems()) {
            renderer.row(meal.getDescription(), money(meal.getPrice()));
        }
        renderer.separator();
        renderer.row("Subtotal", money(order.getSubtotal()));
        renderer.row("Discount: " + order.getDiscountStrategy().getName(), "-" + money(order.getDiscount()));
        renderer.total("Total", money(order.getFinalTotal()));
        renderer.separator();
        renderer.text("Paid by: " + order.getPaymentMethod());
        renderer.text("Thank you! Come again :)");
        return renderer.end();
    }
}
```

A.7 Facade (Kuandyk Koshanov)

facade/RestaurantFacade.java

```java
package restaurant.facade;

import restaurant.bridge.Receipt;
import restaurant.builder.Order;
import restaurant.decorator.Topping;
import restaurant.factory.Meal;
import restaurant.factory.MealCategory;
import restaurant.factory.MealFactory;
import restaurant.observer.CashierDashboard;
import restaurant.observer.CustomerNotifier;
import restaurant.observer.KitchenDisplay;
import restaurant.observer.OrderStatus;
import restaurant.strategy.DiscountStrategy;
import restaurant.strategy.PaymentStrategy;

import java.util.ArrayList;
import java.util.Collections;
```

```java
import java.util.List;

/**
 * FACADE pattern.
 *
 * One simple entry point to the whole restaurant subsystem. The client
 * (console UI) does not need to know about factories, decorators, builders,
 * observers, strategies or receipt renderers - it just calls
 * orderMeal(), placeOrder(), nextStatus(), checkout() and printReceipt().
 *
 * Author: Kuandyk Koshanov (Student 3)
 */
public class RestaurantFacade {
    private final KitchenDisplay kitchen = new KitchenDisplay();
    private final CustomerNotifier customerNotifier = new CustomerNotifier();
    private final CashierDashboard cashier = new CashierDashboard();
    private final List<Order> orders = new ArrayList<>();

    /** Factory Method + Decorator: creates a meal and wraps it with toppings. */
    public Meal orderMeal(MealCategory category, Topping... toppings) {
        Meal meal = MealFactory.forCategory(category).prepareMeal();
        for (Topping topping : toppings) {
            meal = topping.applyTo(meal);
        }
        return meal;
    }

    /** Builder (built by the caller) + Observer: subscribes all observers and places the order. */
    public Order placeOrder(Order.Builder builder) {
        Order order = builder.build();
        order.attach(kitchen);
        order.attach(customerNotifier);
        order.attach(cashier);
        orders.add(order);
        System.out.println("  [Facade] Order #" + order.getId() + " placed for " + order.getCustomerName());
        order.setStatus(OrderStatus.PLACED);
        return order;
    }

    /** Moves the order to the next kitchen status; observers are notified. */
    public void nextStatus(Order order) {
        OrderStatus current = order.getStatus();
        if (current == OrderStatus.SERVED || current == OrderStatus.PAID || current == OrderStatus.CANCELLED) {
            System.out.println("  [Facade] Order #" + order.getId() + " is already " + current);
            return;
        }
        order.setStatus(current.next());
    }

    /** Runs the full kitchen cycle: PLACED -> COOKING -> READY -> SERVED. */
    public void cookAndServe(Order order) {
        while (order.getStatus() != OrderStatus.SERVED
                && order.getStatus() != OrderStatus.PAID
                && order.getStatus() != OrderStatus.CANCELLED) {
            nextStatus(order);
        }
    }

    public void cancel(Order order) {
        if (order.getStatus() == OrderStatus.PAID) {
            System.out.println("  [Facade] Paid order cannot be cancelled");
            return;
        }
        order.setStatus(OrderStatus.CANCELLED);
    }

    /** Strategy: applies the chosen discount and pays with the chosen method. */
    public boolean checkout(Order order, DiscountStrategy discount, PaymentStrategy payment) {
        if (order.getStatus() == OrderStatus.PAID) {
            System.out.println("  [Facade] Order #" + order.getId() + " is already paid");
            return false;
        }
        if (order.getStatus() == OrderStatus.CANCELLED) {
            System.out.println("  [Facade] Order #" + order.getId() + " is cancelled");
            return false;
        }
        order.setDiscountStrategy(discount);
        System.out.printf("  [Facade] Subtotal %.0f, discount \"%s\" -%.0f, to pay %.0f KZT%n",
                order.getSubtotal(), discount.getName(), order.getDiscount(), order.getFinalTotal());
        if (!payment.pay(order.getFinalTotal())) {
            return false;
        }
        order.setPaymentMethod(payment.getName());
        order.setStatus(OrderStatus.PAID);
        return true;
    }

    /** Bridge: any receipt type with any renderer. */
    public String printReceipt(Order order, Receipt receipt) {
        return receipt.print(order);
    }
```

```
    public List<Order> getOrders() {
        return Collections.unmodifiableList(orders);
    }

    public Order findOrder(int id) {
        for (Order order : orders) {
            if (order.getId() == id) {
                return order;
            }
        }
        return null;
    }

    public CashierDashboard getCashier() {
        return cashier;
    }
}
```

A.8 Console client (team)

Main.java

```
package restaurant;

import restaurant.bridge.ConsoleRenderer;
import restaurant.bridge.DetailedReceipt;
import restaurant.bridge.HtmlRenderer;
import restaurant.bridge.Receipt;
import restaurant.bridge.ReceiptRenderer;
import restaurant.bridge.ShortReceipt;
import restaurant.builder.Order;
import restaurant.decorator.Topping;
import restaurant.facade.RestaurantFacade;
import restaurant.factory.Meal;
import restaurant.factory.MealCategory;
import restaurant.strategy.CardPayment;
import restaurant.strategy.CashPayment;
import restaurant.strategy.DiscountStrategy;
import restaurant.strategy.HappyHourDiscount;
import restaurant.strategy.KaspiQrPayment;
import restaurant.strategy.NoDiscount;
import restaurant.strategy.PaymentStrategy;
import restaurant.strategy.StudentDiscount;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

/**
 * Restaurant Ordering System - console client.
 *
 * Talks to the system ONLY through RestaurantFacade.
 * Run with "--demo" to execute a scripted scenario that shows all 7 patterns.
 *
 * Team: Yeskendir Abraimov (Factory Method, Decorator),
 *       Dias Zhumakhan (Observer, Strategy),
 *       Kuandyk Koshanov (Builder, Bridge, Facade)
 */
public class Main {
    private static final RestaurantFacade restaurant = new RestaurantFacade();
    private static final Scanner in = new Scanner(System.in);

    public static void main(String[] args) {
        if (args.length > 0 && args[0].equals("--demo")) {
            runDemo();
            return;
        }
        banner();
        while (true) {
            System.out.println();
            System.out.println("================ MAIN MENU ================");
            System.out.println(" 1. Show menu");
            System.out.println(" 2. Create new order");
            System.out.println(" 3. List orders");
            System.out.println(" 4. Move order to next status");
            System.out.println(" 5. Checkout (discount + payment + receipt)");
            System.out.println(" 6. Cancel order");
            System.out.println(" 7. Run full demo scenario");
            System.out.println(" 0. Exit");
            int choice = readInt("Choose: ", 0, 7);
            try {
                switch (choice) {
                    case 1: showMenu(); break;
                    case 2: createOrder(); break;
                    case 3: listOrders(); break;
```

```
                case 4: nextStatus(); break;
                case 5: checkout(); break;
                case 6: cancelOrder(); break;
                case 7: runDemo(); break;
                case 0:
                    System.out.println("Bye!");
                    return;
                default: break;
            }
        } catch (IllegalStateException e) {
            System.out.println("  [Error] " + e.getMessage());
        }
    }
}

// ============================================================ MENU ACTIONS
private static void banner() {
    System.out.println("+-------------------------------------------------+");
    System.out.println("|      DESIGN PATTERNS BISTRO - Ordering System  |");
    System.out.println("|  Factory | Decorator | Observer | Strategy     |");
    System.out.println("|  Builder | Bridge    | Facade                   |");
    System.out.println("+-------------------------------------------------+");
}

private static void showMenu() {
    System.out.println("---------------- MENU ----------------");
    System.out.println(" Meals:");
    System.out.println("   1. Beef Burger ........ 2500 KZT");
    System.out.println("   2. Margherita Pizza ... 3800 KZT");
    System.out.println("   3. Caesar Salad ....... 2200 KZT");
    System.out.println("   4. Lemonade ........... 900 KZT");
    System.out.println(" Toppings:");
    Topping[] toppings = Topping.values();
    for (int i = 0; i < toppings.length; i++) {
        System.out.printf("   %d. %-18s +%.0f KZT%n", i + 1, toppings[i].getLabel(), toppings[i].getPrice());
    }
}

private static void createOrder() {
    Order.Builder builder = new Order.Builder();
    builder.customer(readLine("Customer name: "));
    int type = readInt("1 - Dine-in, 2 - Delivery: ", 1, 2);
    if (type == 1) {
        builder.dineIn(readInt("Table number: ", 1, 50));
    } else {
        builder.delivery(readLine("Delivery address: "));
    }
    showMenu();
    MealCategory[] categories = MealCategory.values();
    while (true) {
        int m = readInt("Add meal (1-4, 0 = finish): ", 0, 4);
        if (m == 0) {
            break;
        }
        List<Topping> chosen = new ArrayList<>();
        while (true) {
            int t = readInt("  Add topping (1-4, 0 = none/finish): ", 0, Topping.values().length);
            if (t == 0) {
                break;
            }
            chosen.add(Topping.values()[t - 1]);
        }
        Meal meal = restaurant.orderMeal(categories[m - 1], chosen.toArray(new Topping[0]));
        builder.addItem(meal);
        System.out.println("  Added: " + meal);
    }
    String comment = readLine("Comment (Enter to skip): ");
    if (!comment.isEmpty()) {
        builder.comment(comment);
    }
    restaurant.placeOrder(builder);
}

private static void listOrders() {
    if (restaurant.getOrders().isEmpty()) {
        System.out.println("  No orders yet.");
        return;
    }
    for (Order order : restaurant.getOrders()) {
        System.out.println("  " + order);
    }
}

private static Order pickOrder() {
    listOrders();
    if (restaurant.getOrders().isEmpty()) {
        return null;
    }
    Order order = restaurant.findOrder(readInt("Order id: ", 1, Integer.MAX_VALUE));
    if (order == null) {
        System.out.println("  Order not found");
    }
```

```
        return order;
    }

    private static void nextStatus() {
        Order order = pickOrder();
        if (order != null) {
            restaurant.nextStatus(order);
        }
    }

    private static void cancelOrder() {
        Order order = pickOrder();
        if (order != null) {
            restaurant.cancel(order);
        }
    }

    private static void checkout() {
        Order order = pickOrder();
        if (order == null) {
            return;
        }
        System.out.println("Discount: 1 - none, 2 - student -10%, 3 - happy hour (drinks -50%)");
        DiscountStrategy discount;
        switch (readInt("Choose discount: ", 1, 3)) {
            case 2: discount = new StudentDiscount(); break;
            case 3: discount = new HappyHourDiscount(); break;
            default: discount = new NoDiscount();
        }
        order.setDiscountStrategy(discount);
        System.out.printf("  To pay: %.0f KZT%n", order.getFinalTotal());
        System.out.println("Payment: 1 - cash, 2 - card, 3 - Kaspi QR");
        PaymentStrategy payment;
        switch (readInt("Choose payment: ", 1, 3)) {
            case 1: payment = new CashPayment(readInt("Cash given: ", 0, Integer.MAX_VALUE)); break;
            case 2: payment = new CardPayment(readLine("Card number (16 digits): ")); break;
            default: payment = new KaspiQrPayment(readLine("Phone number: "));
        }
        if (!restaurant.checkout(order, discount, payment)) {
            return;
        }
        int kind = readInt("Receipt: 1 - short, 2 - detailed: ", 1, 2);
        int format = readInt("Format: 1 - console, 2 - HTML file: ", 1, 2);
        ReceiptRenderer renderer = format == 1 ? new ConsoleRenderer() : new HtmlRenderer();
        Receipt receipt = kind == 1 ? new ShortReceipt(renderer) : new DetailedReceipt(renderer);
        output(order, restaurant.printReceipt(order, receipt), format == 2);
    }

    private static void output(Order order, String text, boolean html) {
        if (!html) {
            System.out.print(text);
            return;
        }
        Path file = Paths.get("receipt_" + order.getId() + ".html");
        try {
            Files.write(file, text.getBytes(StandardCharsets.UTF_8));
            System.out.println("  HTML receipt saved to " + file.toAbsolutePath());
        } catch (IOException e) {
            System.out.println("  Could not save receipt: " + e.getMessage());
        }
    }

    // ============================================================ DEMO
    private static void section(String title) {
        System.out.println();
        System.out.println("==== " + title + " " + "=".repeat(Math.max(0, 60 - title.length())));
    }

    private static void runDemo() {
        banner();

        section("1. FACTORY METHOD + DECORATOR (Yeskendir Abraimov)");
        Meal burger = restaurant.orderMeal(MealCategory.BURGER, Topping.EXTRA_CHEESE, Topping.BACON);
        Meal pizza = restaurant.orderMeal(MealCategory.PIZZA, Topping.SPICY_SAUCE);
        Meal lemonade = restaurant.orderMeal(MealCategory.DRINK, Topping.EXTRA_ICE);
        Meal salad = restaurant.orderMeal(MealCategory.SALAD);
        System.out.println("  Decorated results:");
        System.out.println("    " + burger);
        System.out.println("    " + pizza);
        System.out.println("    " + lemonade);
        System.out.println("    " + salad);

        section("2. BUILDER + FACADE + OBSERVER (Kuandyk Koshanov, Dias Zhumakhan)");
        Order dineIn = restaurant.placeOrder(new Order.Builder()
                .customer("Aruzhan")
                .dineIn(7)
                .addItem(burger)
                .addItem(lemonade)
                .comment("no onions please"));
        System.out.println();
        Order delivery = restaurant.placeOrder(new Order.Builder()
                .customer("Timur")
```

```
                .delivery("Mangilik El 55, Astana")
                .addItem(pizza)
                .addItem(salad)
                .addItem(restaurant.orderMeal(MealCategory.DRINK)));

        System.out.println();
        System.out.println("  Builder validation - order without meals:");
        try {
            new Order.Builder().customer("Empty").dineIn(1).build();
        } catch (IllegalStateException e) {
            System.out.println("  [Builder] rejected: " + e.getMessage());
        }

        section("3. OBSERVER - status changes notify kitchen/customer");
        restaurant.cookAndServe(dineIn);
        System.out.println();
        restaurant.cookAndServe(delivery);

        section("4. STRATEGY - discounts and payments (Dias Zhumakhan)");
        DiscountStrategy[] discounts = {new NoDiscount(), new StudentDiscount(), new HappyHourDiscount()};
        System.out.printf("  Order #%d (subtotal %.0f KZT) with different discount strategies:%n",
                dineIn.getId(), dineIn.getSubtotal());
        for (DiscountStrategy d : discounts) {
            dineIn.setDiscountStrategy(d);
            System.out.printf("    %-26s discount %5.0f -> total %5.0f KZT%n", d.getName(), dineIn.getDiscount(),
dineIn.getFinalTotal());
        }
        System.out.println();
        restaurant.checkout(dineIn, new HappyHourDiscount(), new CashPayment(5000));
        System.out.println();
        System.out.println("  Card payment attempt with a wrong number:");
        restaurant.checkout(delivery, new StudentDiscount(), new CardPayment("1234"));
        System.out.println();
        restaurant.checkout(delivery, new StudentDiscount(), new KaspiQrPayment("+7 701 123 45 67"));

        section("5. BRIDGE - receipt type x output format (Kuandyk Koshanov)");
        System.out.println("  ShortReceipt + ConsoleRenderer:");
        System.out.print(restaurant.printReceipt(dineIn, new ShortReceipt(new ConsoleRenderer())));
        System.out.println("  DetailedReceipt + ConsoleRenderer:");
        System.out.print(restaurant.printReceipt(dineIn, new DetailedReceipt(new ConsoleRenderer())));
        System.out.println("  DetailedReceipt + HtmlRenderer:");
        output(delivery, restaurant.printReceipt(delivery, new DetailedReceipt(new HtmlRenderer())), true);

        section("SUMMARY");
        for (Order order : restaurant.getOrders()) {
            System.out.println("  " + order);
        }
        System.out.printf("  Cashier dashboard: %d paid orders, revenue %.0f KZT%n",
                restaurant.getCashier().getPaidOrders(), restaurant.getCashier().getRevenue());
    }

    // ============================================================== INPUT HELPERS
    private static String readLine(String prompt) {
        System.out.print(prompt);
        if (!in.hasNextLine()) {
            System.exit(0);
        }
        return in.nextLine().trim();
    }

    private static int readInt(String prompt, int min, int max) {
        while (true) {
            String line = readLine(prompt);
            try {
                int value = Integer.parseInt(line);
                if (value >= min && value <= max) {
                    return value;
                }
            } catch (NumberFormatException ignored) {
                // fall through
            }
            System.out.println("  Please enter a number from " + min + " to " + max);
        }
    }
}
```